Improving Prior-Guided Bayesian Optimization through Revisited Decay and Rejection Mechanisms

Kay Bennet Städing

26.08.2026
Version: 1.0

Institute of
Artificial Intelligence

Leibniz
Universität
Hannover

Electrical Engineering and Computer Science,
Institute of Artificial Intelligence (LUHAI)
Welfengarten 1
30167 Hannover

Bachelor's Thesis

Improving Prior-Guided Bayesian Optimization through Revisited Decay and Rejection Mechanisms

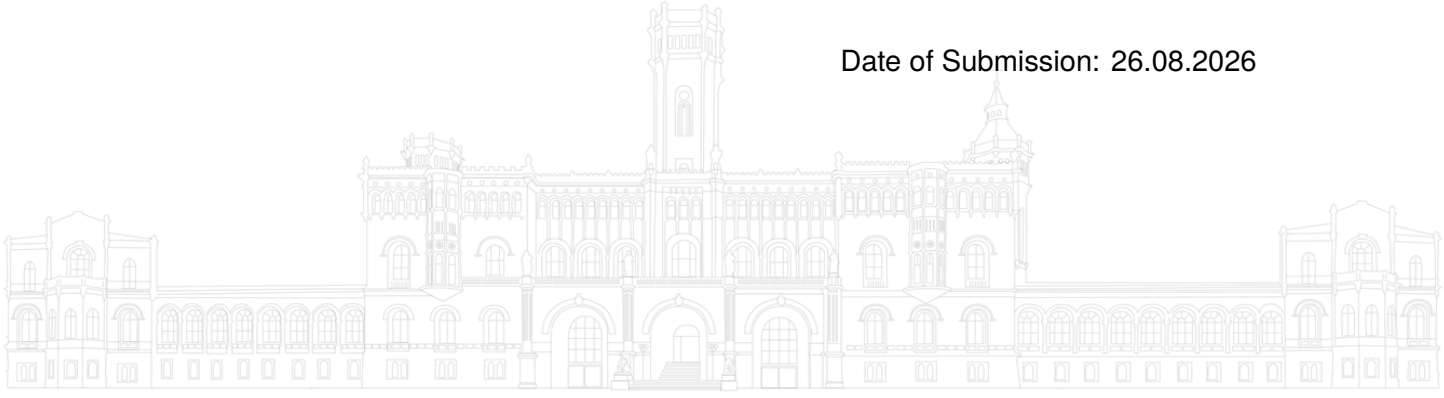
Kay Bennet Städing

1. Reviewer Prof. Dr. rer. nat. Marius Lindauer
Electrical Engineering and Computer Science
Leibniz University Hannover

2. Reviewer Prof. Dr. rer. nat. Henning Wachsmuth
Electrical Engineering and Computer Science
Leibniz University Hannover

Supervisors M.Sc. Lukas Fehring

Date of Submission: 26.08.2026



Kay Bennet Städing
Improving Prior-Guided Bayesian Optimization through Revisited Decay and Rejection Mechanisms
Bachelor's Thesis, 26.08.2026
Reviewers: Prof. Dr. rer. nat. Marius Lindauer
and Prof. Dr. rer. nat. Henning Wachsmuth
Supervisors: M.Sc. Lukas Fehring
Automated Machine Learning (AutoML)
Institute of Artificial Intelligence (LUHAI)
Electrical Engineering and Computer Science
Welfengarten 1
30167 Hannover

Abstract

Bayesian Optimization (BO) is the dominant approach for hyperparameter optimization when faced with expensive black-box objectives. Prior-guided variants such as πBO and DynaBO accelerate the search using a practitioner-specified belief about the optimum. Both methods define the prior-term's influence through hyperparameters fixed before the run, including decay rates, decay schedules and thresholds for accepting priors in the first place. This thesis investigates whether the process can be meaningfully adapted online, by adjusting prior trust based on observations gathered during optimization. We first define oracle variants of decay and rejection that retrospectively select the best fixed setting from a discrete candidate set at every iteration. They reveal significant gaps in regret compared to their respective static baselines and quantify the theoretical room for improvement. Building on this, we propose observation-adaptive mechanisms that close part of this gap: Adaptive decay schedules that scale β and traverse a discrete decay ladder based on a sliding window of recent costs, and a prior discard criterion that re-evaluates already injected priors on a stepwise basis. We conduct our experiments on four PD1 scenarios under four prior-quality kinds with 30 seeds each. Adaptive decay narrows the oracle gap most for expert and deceptive priors, adaptive rejection benefits deceptive priors while permitting a less conservative threshold, and combining both mechanisms yields little beyond stepwise rejection alone, aside from a marked reduction in final regret under expert priors. Overall, the adaptive mechanisms leave near-optimal priors largely unaffected but substantially reduce regret when priors are less informative, indicating that observation-adaptive prior management primarily raises the worst-case performance of prior-guided BO.

Declaration

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst, keine anderen als die angegebenen Quellen und Hilfsmittel verwendet sowie die aus fremden Quellen direkt oder indirekt übernommenen Stellen/Gedanken als solche kenntlich gemacht habe.

Hannover, 26.08.2026, _____

Kay Bennet Städing

Acknowledgment

I thank Lukas Fehring for his supervision throughout this thesis, and for providing the codebase underlying DynaBO (Fehring et al., 2026), which I forked and extended substantially for all experiments reported here (`https://github.com/kaybennet/PriorQuantification_Modifications`). The base experimental pipeline, including the SMAC3 (Lindauer et al., 2022) integration, the py-experimenter/MySQL infrastructure, and the prior-sampling callbacks for the four prior kinds, all originate from this repository.

Consistent with the principles of good academic practice, all use of AI tools in the preparation of this thesis is documented below similarly to other literature and resources, ensuring that the origin of affected formulations or content remains traceable. Each row corresponds to a single instance of use, specified by AI tool and by the part(s) of the thesis affected.

Column "Type of Use": For each pair of AI tool and affected part(s), we specify the type of use with one of these (bold) terms:

- **Outline**/structure creation (e.g., drafting a chapter outline or table of contents)
- **Idea generation**/brainstorming (e.g., exploring possible approaches or arguments)
- **Literature** research (e.g., finding or summarizing relevant sources)
- **Translation** (e.g., translating a passage into another language)
- **Language revision**/grammar revision (e.g., checking spelling, punctuation, ...)
- **Content revision**/rewriting (e.g., rephrasing or restructuring existing text)
- **Summarization** (e.g., condensing a text or source into key points)
- **Code** generation/debugging (e.g., writing or fixing a code snippet)
- **Data analysis** (e.g., interpreting or processing data or results)
- **Other** (any use not covered above)

Column "Implementation": For each pair of AI tool and affected part(s), we specify how we implemented the type of use with one of these (bold) terms:

- **Ideas only, revised:** Only ideas were used as inspiration; the content was substantially revised afterward.
- **First draft, reworked:** Output was used as a first draft and reworked significantly.
- **Reviewed, partly used:** Output was reviewed critically and only partly incorporated.
- **Quick check only:** Output was used for a quick check; no content was adopted.
- **Minor edits:** The output was adopted with only minor edits.
- **Code tested:** Generated code was tested and verified before use.
- **Other:** Any implementation not covered above

#	AI Tool	Version	Affected Part(s)	Type of Use	Implementation
Documentation of AI usage in this thesis:					
1	Claude `https://claude.ai`	Sonnet 5.x	Plotting & Analysis Code	Code	Code tested
2	Claude `https://claude.ai`	Sonnet 5.x	Experiment pipeline debugging	Code	Code tested
3	Claude `https://claude.ai`	Sonnet 5.x	Whole thesis	Language revision	Minor edits
4	Claude `https://claude.ai`	Sonnet 5.x	Chapters 4 and 5	Data Analysis	Quick check only
5	Claude `https://claude.ai`	Sonnet 5.x	Whole thesis	Literature	Ideas only, revised
6	Claude `https://claude.ai`	Sonnet 5.x	Result tables, figure captions	Data analysis	Reviewed, partly used
7	Claude `https://claude.ai`	Sonnet 5.x	Appendix	Summarization	First draft, reworked

Contents

1. Introduction **11**
1.1. Motivation and Problem Statement . . . 11
1.2. Contributions . . . 12

2. Related Work **13**
2.1. Machine Learning . . . 13
2.2. Hyperparameter Optimization . . . 13
2.3. Bayesian Optimization . . . 15
2.3.1. Acquisition Functions . . . 15
2.4. Prior-Augmented Optimization . . . 16
2.4.1. BOPrO . . . 17
2.4.2. πBO . . . 17
2.4.3. Dynamic Priors in Bayesian Optimization . . . 18
2.5. Dynamic Algorithm Configuration of Bayesian Optimization . . . 19
2.5.1. Switching and Weighting Acquisition Functions . . . 19
2.5.2. The Remaining Gap . . . 19

3. Approach **20**
3.1. Adaptive Prior Decay for a Single Injected Prior . . . 20
3.1.1. Motivation . . . 20
3.1.2. Generalized Decay Schedules . . . 22
3.1.3. Oracle Decay . . . 24
3.1.4. Observation-Adaptive Decay . . . 24
3.2. Adaptive Prior Rejection for Multiple Injected Priors . . . 27
3.2.1. Fixed Decay Schedule . . . 27
3.2.2. Oracle Rejection . . . 27
3.2.3. Per-Step Reassessment of Accepted Priors . . . 29
3.3. Combined Mechanism . . . 30

4. Experiments **31**
4.1. Experimental Setup . . . 31
4.1.1. Shared Experimental Configuration . . . 32
4.1.2. Statistical Testing Protocol . . . 33
4.1.3. Area over the Curve and Gap Closed . . . 33
4.2. RQ1: Adaptive Decay for πBO . . . 34
4.2.1. Oracle Prior Decay . . . 34
4.2.2. Observation-Adaptive Decay Heuristics . . . 34
4.3. RQ2: Adaptive Rejection for DynaBO . . . 35
4.3.1. Oracle Prior Rejection . . . 35
4.3.2. Shared DynaBO Configuration . . . 35
4.4. RQ3: Combined Mechanism . . . 36
4.4.1. Combined Oracle . . . 36
4.4.2. Combining Rejection Heuristics . . . 36

5. Results **37**
5.1. Observation-Adaptive Prior Decay . . . 37
5.1.1. Analyzing the Prior Decay Improvement Potential . . . 38
5.1.2. Adaptive Decay Schedule (First Heuristic) . . . 39
5.1.3. Adaptive Prior Strength (Second Heuristic) . . . 41
5.1.4. Combined Adaptive Decay and Strength (Third Heuristic) . . . 43
5.1.5. Sensitivity Analysis of the Window Size $\mathcal{W}$ Introduced by Selected Adaptive Heuristics . . . 44
5.1.6. Numerical Comparison of the Adaptive Decay Heuristics . . . 44
5.2. Observation-Adaptive Prior Rejection . . . 46
5.2.1. Analyzing the Oracle Rejection's Improvement Potential . . . 46
5.2.2. First Heuristic: Stepwise Prior Rejection Check . . . 47
5.2.3. Sensitivity Analysis of the Prior Rejection Threshold τ . . . 48
5.2.4. Second Heuristic: Stepwise Prior Rejection Check Using a Looser Threshold . . . 50
5.2.5. Analyzing Frequency of Post-Acceptance Prior Discards . . . 51
5.3. Combined Observation-Adaptive Mechanisms . . . 53
5.3.1. DynaBO with Combined Heuristics . . . 53
5.3.2. Regret Difference of Stepwise and On Injection Prior Rejection Checks 54
5.3.3. Evaluating the Adaptive Rejection Heuristics in Conjunction . . . 55

6. Conclusion **57**
6.1. Answering the Research Questions . . . 57
6.2. Overall . . . 58
6.3. Limitations . . . 59
6.4. Future Work . . . 59
6.5. Concluding Remarks . . . 60

Bibliography **61**

A. Appendix **65**
A.1. Further Experimental Specifications . . . 65
A.1.1. Loss Functions . . . 65
A.1.2. Further Beta Evaluations for DynaBO . . . 66
A.2. πBO: Configuration Heatmap . . . 67
A.3. Full Adaptive Decay Results . . . 68
A.3.1. Adaptive-Schedule and Adaptive-Strength Results . . . 68
A.3.2. Combined Decay Results and Window-Size Sensitivity . . . 71
A.4. Full Adaptive Rejection Results . . . 73
A.5. Combined Mechanisms: Full Results . . . 77

1 Introduction

The performance of a machine learning (ML) model is critically shaped by a variety of factors, including its hyperparameters. Hyperparameters include configuration choices that influence the training process but are not learned from data (Feurer et al., 2019). Selecting well-performing configurations manually is often becoming increasingly infeasible as models and search spaces grow in complexity, which motivates more sophisticated approaches to automate the search (Feurer et al., 2019; Yang et al., 2020). Among the available approaches to Hyperparameter Optimization (HPO), Bayesian Optimization (BO) has established itself as a dominant paradigm for expensive blackbox objectives (Shahriari et al., 2016). BO can be improved further by incorporating prior knowledge that practitioners often already have about promising regions of the search space, an idea that has motivated a growing body of work on prior-guided BO (Hvarfner et al., 2022). This thesis investigates whether the trust placed in such prior knowledge can be managed adaptively during optimization rather than being fixed in advance, and proposes observation-driven mechanisms to achieve this.

1.1 Motivation and Problem Statement

Although BO on average outperforms uninformed strategies, it requires an initial set of evaluated configurations to fit its surrogate model, consuming part of the limited evaluation budget before the optimization itself can begin (Feurer et al., 2019). Once this initial phase concludes, standard BO treats every region of the search space as equally promising. Methods such as πBO (Hvarfner et al., 2022) and *DynaBO* (Fehring et al., 2026) address this by incorporating a user-specified prior over the location of the optimum directly into the acquisition function, so that the surrogate-guided phase of the search is informed by prior knowledge from the moment it begins.

In practice, however, specifying a genuinely reliable prior is itself difficult. Even experienced practitioners may produce beliefs that range from highly informative to actively misleading (Hvarfner et al., 2022). Both πBO and DynaBO account for this by managing how strongly the prior is trusted over the course of the run. However, the parameters surrounding this trust, a decay rate in πBO and a rejection threshold in DynaBO, are fixed before optimization begins (Hvarfner et al., 2022; Fehring et al., 2026). A setting chosen to fully exploit an informative prior for example is precisely what allows a misleading one to derail the search. Because acceleration and robustness are in direct tension, no single static setting can be optimal across the full range of prior qualities (Fehring et al., 2026).

1.2 Contributions

This raises a natural question: Can the trust placed in a prior be adjusted online, based on the evidence and observations that accumulate during the run? We examine the rate at which the prior loses influence over time (decay) and ways to discard the prior altogether (rejection). Concretely, this thesis investigates the following research questions (RQs):

RQ1 How does πBO's fixed decay schedule compare to Vanilla BO and an oracle decay (i.e., the best possible decay from a discrete set of options) across varying prior qualities, and can observation-adaptive alternatives narrow this gap?

RQ2 How does DynaBO's fixed rejection threshold compare to Vanilla BO and an oracle threshold across varying prior qualities, and can observation-adaptive alternatives narrow this gap?

RQ3 Does combining the modified prior decay schedule and rejection criterion result in further improvement beyond what either mechanism achieves independently?

To address these questions, we propose observation-adaptive alternatives to the static prior-management mechanisms of prior decay and prior rejection[1]: The fixed decay schedule is replaced with one that adjusts online based on signals derived from the observed objective values, and for prior rejection, the static threshold is replaced with a criterion that is re-evaluated at each iteration using analogous observational evidence. Both mechanisms are also combined to assess whether they yield complementary gains. For each mechanism we additionally define an oracle variant, i.e., the oracle decay and the oracle rejection, which select the best-performing fixed setting in hindsight from a discrete candidate set and serve as empirical upper bounds to quantify the theoretical room for improvement. All variants are evaluated on established HPO surrogate benchmarks across priors of varying quality, ranging from highly informative to adversarial. In chapter 2 we review the foundations of hyperparameter optimization including BO and several of its prior-guided variants. Then chapter 3 presents the proposed observation-adaptive mechanisms and their oracle counterparts. Lastly chapter 4 and chapter 5 describe the experimental setup and results, and chapter 6 concludes with a summary and possible directions for future work.

[1] Implementation available at `https://github.com/kaybennet/PriorQuantification_Modifications`.

2 Related Work

This chapter introduces the background and related work of this thesis. We first briefly review the distinction between machine learning parameters and hyperparameters in section 2.1, before turning to *Hyperparameter Optimization* (HPO) and some of its most prominent methods in section 2.2, and *Bayesian Optimization* (BO) and its key components in section 2.3. Then, section 2.4 discusses how user-specified priors can be incorporated into BO, focusing on πBO (Hvarfner et al., 2022) and DynaBO (Fehring et al., 2026), the two methods this thesis builds and extends upon. Finally, section 2.5 reviews Dynamic Algorithm Configuration and its prior applications to acquisition functions, identifying the gap in adaptive prior trust that motivates this thesis's research questions.

2.1 Machine Learning

Machine learning (ML) studies algorithms that learn patterns from data rather than following explicit rules. In supervised machine learning, a model is trained on labeled data to generalize to unseen examples by fitting internal parameters on the training data by minimizing a loss function. Unsupervised learning algorithms on the other hand are used to find patterns from unlabeled data (Yang et al., 2020).

Beyond these learned parameters, every ML algorithm can also be configured using *hyperparameters*. These include, but are not limited to the learning rate, batch size, or regularization strength. They control the training process but must be set beforehand rather than learned from data (Feurer et al., 2019). In addition to being real-valued, hyperparameters can also for example be integer-valued, binary, or categorical. Since they strongly affect the resulting model's performance, choosing appropriate hyperparameters is essential for good generalization, motivating the several methods for tuning discussed next.

2.2 Hyperparameter Optimization

Automating the search for appropriate hyperparameter configurations reduces the human effort required to apply machine learning, improves performance by tailoring algorithms to the problem at hand, and improves the reproducibility and fairness of scientific studies (Feurer et al., 2019).

We denote A formally as a machine learning algorithm with a set $H = \{h_1, \ldots, h_N\}$ of $|H| = N$ hyperparameters. Each hyperparameter $h \in H$ has its own domain Λ_h. We then define the configuration space as the Cartesian product $\Lambda = \Lambda_1 \times \Lambda_2 \times \cdots \times \Lambda_N$. Given a

loss function $\mathcal{L} : \Lambda \to \mathbb{R}$ and a distribution $\mathcal{D}$ over training/validation splits, the objective of HPO is to find the configuration that minimizes the expected loss over splits drawn from $\mathcal{D}$:

$$\lambda^* \in \arg\min_{\lambda \in \Lambda} \mathbb{E}_{(D_{\text{train}}, D_{\text{valid}}) \sim \mathcal{D}} \left[\mathcal{V}(L, A_\lambda, D_{\text{train}}, D_{\text{valid}}) \right], \tag{2.1}$$

where $\mathcal{V}(L, A_\lambda, D_{\text{train}}, D_{\text{valid}})$ measures the loss $\mathcal{L}$ of the model produced by A_λ when trained on D_{train} and evaluated on D_{valid} (Feurer et al., 2019). As multiple configurations may attain the minimum, λ^* need not be a unique solution.

Grid Search Any blackbox optimization method can be applied to the optimization problem outlined in Equation 2.1. One of the most straightforward approaches is *grid search*, which evaluates all combinations from a pre-specified finite set of values per hyperparameter. While conceptually simple, the number of evaluations does grow exponentially in the number of hyperparameters N. Therefore, under a fixed budget B only $B^{1/N}$ distinct values can be placed along each dimension. Increasing the resolution of discretization to better approximate continuous domains further compounds this cost, as every additional value per dimension multiplies the total number of evaluations (Feurer et al., 2019).

Random Search This method instead samples configurations independently at random until the budget is exhausted, so for continuous hyperparameters it explores a distinct value along each dimension in every evaluation (Feurer et al., 2019). Given a budget B, random search is able to explore B different configurations along each axis. Since hyperparameters often differ in importance, random search effectively devotes more samples to the influential dimensions, thus yielding better results than grid search under the same budget (Bergstra et al., 2012). See Figure 2.1 for an illustration comparing both methods when tasked with finding the maximum of a function over a two-dimensional domain.

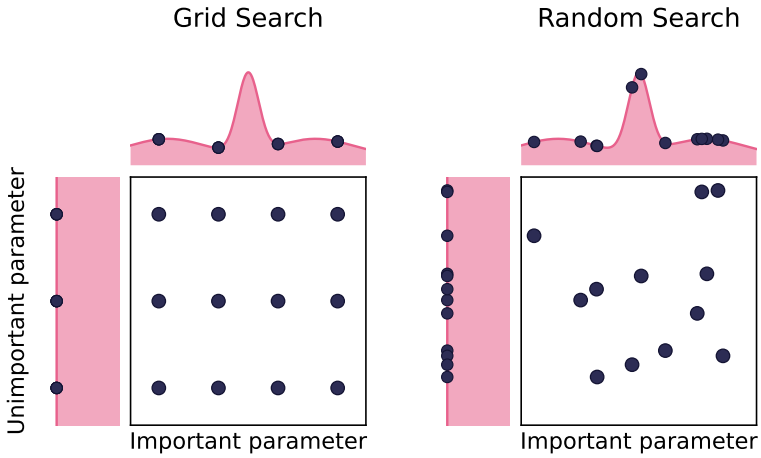


Figure 2.1.: Comparison of grid search and random search for maximizing a function with one important and one unimportant parameter. Custom illustration based on the works of Bergstra et al. (2012) and Feurer et al. (2019).

Despite this advantage, both methods remain fundamentally sample-inefficient: Each evaluation is conducted independently, with no knowledge transferred from prior observations. Since a single function evaluation in HPO typically requires training a full machine learning model, which can take hours or even days for large models or datasets (Feurer et al., 2019), this inefficiency motivates more principled approaches that model the objective function and use past evaluations to guide future ones.

2.3 Bayesian Optimization

Bayesian Optimization (BO) is a sequential, model-based strategy for optimizing expensive blackbox functions under a limited evaluation budget (Shahriari et al., 2016). As introduced in Equation 2.1, the objective function maps a configuration $\lambda \in \Lambda$ to a real-valued loss. The central idea of BO is to build a probabilistic, cheap-to-evaluate surrogate model of this objective from the observations gathered so far and use it to decide which configuration to evaluate next. We call the best configuration observed so far the incumbent $\hat{\lambda}$. Unlike grid or random search, BO uses the results of previous evaluations to guide future ones, making it substantially more sample-efficient and thus particularly well-suited for the expensive evaluations typical of HPO (Shahriari et al., 2016).

BO begins with an *initial design*: a set of M configurations $\mathcal{D}_0 = \{(\lambda_i, y_i)\}_{i=1}^{M}$, typically obtained from a space-filling sampling strategy such as random sampling or a low-discrepancy Sobol sequence (Shahriari et al., 2016), which aims to cover the hyperparameter configuration space diversely before any model-guided search begins. Throughout this thesis, we use a Sobol design. These initial observations give the surrogate a diverse first estimate of the objective. BO then proceeds iteratively: At iteration n, a probabilistic surrogate model $\hat{f}$ is fitted to all observations $\mathcal{D}_n$, the next candidate is obtained by maximizing the acquisition function, $\lambda_{n+1} \in \arg\max_{\lambda \in \Lambda} \alpha_{\hat{f}}(\lambda; \mathcal{D}_n)$, that candidate is evaluated on the black-box objective to yield y_{n+1}, and the pair is appended to the dataset, $\mathcal{D}_{n+1} = \mathcal{D}_n \cup \{(\lambda_{n+1}, y_{n+1})\}$. This loop continues until the evaluation budget is exhausted. Algorithm 1 summarizes the procedure.

Algorithm 1 Bayesian Optimization

1: Sample initial design $\mathcal{D}_0 = \{(\lambda_i, y_i)\}_{i=1}^{M}$
2: **for** $n = 1, 2, 3, \ldots$ **do**
3: Fit surrogate model to $\mathcal{D}_n$
4: Select $\lambda_{n+1} = \arg\max_{\lambda \in \Lambda} \alpha(\lambda; \mathcal{D}_n)$
5: Evaluate objective to obtain y_{n+1}
6: Augment data $\mathcal{D}_{n+1} = \mathcal{D}_n \cup \{(\lambda_{n+1}, y_{n+1})\}$
7: **end for**

2.3.1 Acquisition Functions

The acquisition function $\alpha_{\hat{f}} : \Lambda \to \mathbb{R}$ is used to pick appropriate candidates based on the exploration exploitation tradeoff. At iteration n, the surrogate model conditioned on the observations $\mathcal{D}_n$ gathered so far yields, for any candidate $\lambda \in \Lambda$, a posterior predictive

mean $\mu_n(\lambda)$ and standard deviation $\sigma_n(\lambda)$, which the acquisition functions below combine in different ways. Throughout this section, Φ and ϕ denote the cumulative distribution function (CDF) and probability density function (PDF) of the standard normal distribution, respectively. We state acquisition functions in maximization form. Signs invert for minimization. Also, note that there is not a single universally optimal acquisition function. We outline some options in the following paragraphs.

Probability of Improvement (PI) Given an incumbent target τ_{inc}, typically set to the best value observed so far, Probability of Improvement (PI) measures the probability, under the surrogate's posterior predictive distribution, that a candidate λ improves upon τ_{inc} (Shahriari et al., 2016):

$$\alpha_{\text{PI}}(\lambda; \mathcal{D}_n) := \Phi\left(\frac{\mu_n(\lambda) - \tau_{\text{inc}}}{\sigma_n(\lambda)}\right). \tag{2.2}$$

Because this criterion only cares whether an improvement occurs and not how large it is, PI tends to exploit quite aggressively (Shahriari et al., 2016).

Expected Improvement (EI) Expected Improvement addresses this by additionally weighting the size of the improvement (Shahriari et al., 2016):

$$\alpha_{\text{EI}}(\lambda; \mathcal{D}_n) = \big(\mu_n(\lambda) - f(\hat{\lambda})\big)\, \Phi\big(z_n(\lambda)\big) + \sigma_n(\lambda)\, \phi\big(z_n(\lambda)\big) \tag{2.3}$$

where $z_n(\lambda) := \big(\mu_n(\lambda) - f(\hat{\lambda})\big)/\sigma_n(\lambda)$ is the standardized improvement over the incumbent. This often yields a more balanced results than PI and makes EI the most widely used acquisition function in practice (Shahriari et al., 2016).

Upper Confidence Bound (UCB) Upper Confidence Bound instead follows an optimism-under-uncertainty principle by directly combining the posterior mean and standard deviation (Shahriari et al., 2016):

$$\alpha_{\text{UCB}}(\lambda; \mathcal{D}_n) := \mu_n(\lambda) + \beta_n \sigma_n(\lambda). \tag{2.4}$$

Its minimization counterpart, Lower Confidence Bound (LCB), is used later in this thesis as a safeguard acquisition function to assess the usefulness of injected priors. We discuss priors and their benefits to BO in the following section.

2.4 Prior-Augmented Optimization

Despite BO's sample efficiency, its adoption among practitioners remains limited: A survey of tuning practices at NeurIPS 2019 and ICLR 2020, reported by Bouthillier et al. (2020), found manual search to be the most prevalent method at the time, with Grid Search and Random Search following right after. As practitioners develop deeper familiarity with their models, they accumulate informed beliefs about which regions of the configuration space are likely to yield good results, for instance, that the learning rate of Adam is often best in the vicinity of 1×10^{-3} (Hvarfner et al., 2022; Kingma et al., 2014). Standard BO (Vanilla BO)

has no mechanism to incorporate such beliefs, which Hvarfner et al. (2022) identify as a key reason practitioners continue to prefer manual search despite its documented inefficiency.

2.4.1 BOPrO

An early and influential approach to incorporating user beliefs about the optimum into BO is BOPrO (Souza et al., 2021). BOPrO allows practitioners to specify a prior distribution $P_g(\boldsymbol{\lambda})$ over regions of the input space expected to contain good configurations. To make this prior compatible with a standard surrogate model, BOPrO translates the predictive distribution $p(y \mid \boldsymbol{\lambda})$ into a probability-of-improvement quantity that captures how likely a given input is to produce a function value below the γ-quantile f_γ of all observations collected so far:

$$\mathcal{M}_g(\boldsymbol{\lambda}) = p\big(f(\boldsymbol{\lambda}) < f_\gamma \mid \boldsymbol{\lambda}, \mathcal{D}_t\big) = \Phi\left(\frac{f_\gamma - \mu_{\boldsymbol{\lambda}}}{\sigma_{\boldsymbol{\lambda}}}\right), \tag{2.5}$$

where $\mu_{\boldsymbol{\lambda}}$ and $\sigma_{\boldsymbol{\lambda}}$ are the predictive mean and standard deviation of the surrogate at $\boldsymbol{\lambda}$, and Φ denotes the standard normal CDF (Souza et al., 2021). This model-derived quantity is then combined with the user prior into a pseudo-posterior:

$$g(\boldsymbol{\lambda}) \propto P_g(\boldsymbol{\lambda}) \cdot \mathcal{M}_g(\boldsymbol{\lambda})^{t/\beta}, \tag{2.6}$$

where t is the current iteration and β controls the balance between prior and model (Souza et al., 2021). As optimization progresses, the exponent t/β grows and the data-driven model $\mathcal{M}_g(\boldsymbol{\lambda})$ gradually dominates the user prior $P_g(\boldsymbol{\lambda})$. Souza et al. (2021) show that in the limit the acquisition function reduces to maximizing the probability of improvement on the surrogate alone, enabling recovery from misleading priors.

Despite this desirable prior-forgetting property, BOPrO has several limitations. Its construction relies on a reformulation of Expected Improvement, which ties the method to a single acquisition function and prevents straightforward combination with alternatives such as UCB. Furthermore, BOPrO does not provide formal convergence guarantees. Finally, both the prior and the decay schedule t/β must be fixed before optimization begins with no possibility of revision during the run. These constraints lead directly into the more flexible and theoretically grounded approach introduced by πBO (Hvarfner et al., 2022).

2.4.2 πBO

Building on BOPrO, πBO (Hvarfner et al., 2022) extends the idea of introducing a user-specified probability distribution $\pi(\lambda)$ over the configuration space, representing beliefs about the location of the optimum. Rather than modifying the surrogate model, πBO weights the acquisition function directly by $\pi(\lambda)$, biasing candidate selection toward high-probability regions under the prior.

To ensure that prior influence diminishes as the surrogate accumulates observations, the weighting is equipped with a decaying exponent β/n, where β is a user-specified confidence parameter and n is the current iteration count:

$$\alpha_{\pi,n}(\lambda, \mathcal{D}_n) \coloneqq \alpha(\lambda, \mathcal{D}_n) \cdot \pi(\lambda)^{\beta/n}. \tag{2.7}$$

As n grows, $\pi(\lambda)^{\beta/n} \to 1$ and the method starts resembling Vanilla BO again, providing theoretical robustness against falsely specified priors.

A key practical strength of πBO is its generality: It is agnostic to both the surrogate model and the choice of acquisition function, and requires only a single multiplicative modification to the standard BO loop (Hvarfner et al., 2022).

However, the prior must be fully specified before optimization begins and cannot be revised once it has been injected. This means that any knowledge a practitioner gains during optimization or any initial misjudgment cannot be acted upon, inspiring more dynamic approaches. Prior guidance has also been extended along a different axis, integrating a user prior into multi-fidelity, Hyperband-style optimization (Mallik et al., 2023). This thesis instead remains in the single-fidelity setting and focuses on adaptive trust dynamics.

2.4.3 Dynamic Priors in Bayesian Optimization

πBO's limitation of a single fixed prior is addressed by DynaBO (Fehring et al., 2026), which generalizes the prior-weighting framework to a finite sequence of priors $\{\pi^{(m)}\}_{m=1}^{M}$ that can be injected at arbitrary times $\{t^{(m)}\}_{m=1}^{M}$ during optimization. This enables practitioners to continuously steer the search as their understanding of the problem evolves. Each prior contributes independently to the acquisition function, weighted by its age. This means that older priors exert less influence than recently provided ones, with the decay controlled by a monotone decay function d applied to the elapsed time since injection $t - t^{(m)}$:

$$\alpha_{\hat{f}}^{\text{dyna}}(\lambda) \coloneqq \alpha_{\hat{f}}(\lambda) \cdot \sum_{m=1}^{M} \pi^{(m)}(\lambda)^{\beta/d(t-t^{(m)})}. \tag{2.8}$$

Like πBO, DynaBO is agnostic to both the surrogate model and the acquisition function.

Rejecting Priors at Injection

As the ability to inject priors at any point increases the risk of misleading inputs, DynaBO additionally proposes a data-driven safeguard for prior acceptance. Before a new prior $\pi^{(m)}$ is incorporated, the surrogate model $\hat{f}$ is used to compare the expected acquisition value under configurations sampled from the prior's distribution $F_{\pi^{(m)}}$ against that under configurations sampled from a normal distribution $\mathcal{N}_{\hat{\lambda}}$ centered on the current incumbent $\hat{\lambda}$ (Fehring et al., 2026).

The prior is accepted if and only if this difference exceeds a threshold τ:

$$\mathbb{E}_{\lambda \sim F_{\pi(m)}}\left[\xi_{\hat{f}}(\lambda)\right] - \mathbb{E}_{\lambda \sim \mathcal{N}_{\hat{\lambda}}}\left[\xi_{\hat{f}}(\lambda)\right] \geq \tau. \tag{2.9}$$

Here ξ denotes the acquisition function used for the safeguard, which can differ from the main acquisition function α; Fehring et al. (2026) recommend Lower Confidence Bound (LCB) to assess a prior's remaining optimization potential. A higher value of τ enforces stricter filtering, rejecting a larger proportion of priors including misleading ones, at the cost of also discarding marginally informative ones.

2.5 Dynamic Algorithm Configuration of Bayesian Optimization

Dynamic Algorithm Configuration (DAC) treats an algorithm's hyperparameters not as a fixed setting but as a policy that adapts over the course of a run (Biedenkapp et al., 2020). Two recent works apply this idea to components of the BO loop itself.

2.5.1 Switching and Weighting Acquisition Functions

Benjamins et al. (2022) train a random forest on landscape features of the initial design to select or switch acquisition functions per run, outperforming any static choice on their used benchmark suite.

Benjamins et al. (2023) instead adapt a single acquisition function online: A mixing parameter that controls the exploration-exploitation weighting within Expected Improvement is adjusted continuously based on a signal derived from optimization progress. Together, these works show that both the choice and internal configuration of an acquisition function can be treated as a policy that responds to observations gathered during the run, instead of a statically defined decision.

2.5.2 The Remaining Gap

Despite these advances, both the decay function d in DynaBO (and the analogous exponent schedule β/n in πBO) and the rejection threshold τ in DynaBO are fixed before optimization begins, independent of the observations accumulated during the run. This limits their ability to respond to the actual informativeness of a prior as we gather further evidence. Benjamins et al. (2022) and Benjamins et al. (2023) demonstrate that dynamic algorithm configuration is effective within the acquisition function. Now, this thesis investigates whether the same principle applies to prior trust, a component of the BO loop that has so far primarily remained statically configured.

3 Approach

This chapter proceeds as follows: First, section 3.1 defines an empirical upper bound (oracle) for prior decay and derives a family of observation-adaptive decay heuristics to narrow the resulting gap. Then, section 3.2 extends this construction to DynaBO's rejection criterion, and section 3.3 examines the combination of both mechanisms. Throughout, the respective oracles denote the best configurations selectable in hindsight from a discrete candidate set, and therefore serve as a strong reference trajectory on what an online strategy could achieve.

3.1 Adaptive Prior Decay for a Single Injected Prior

This section develops the approach used to answer RQ1. It is restricted to one prior injected directly after initial design and therefore makes use of πBO and many of its proposed configurations (Hvarfner et al., 2022).

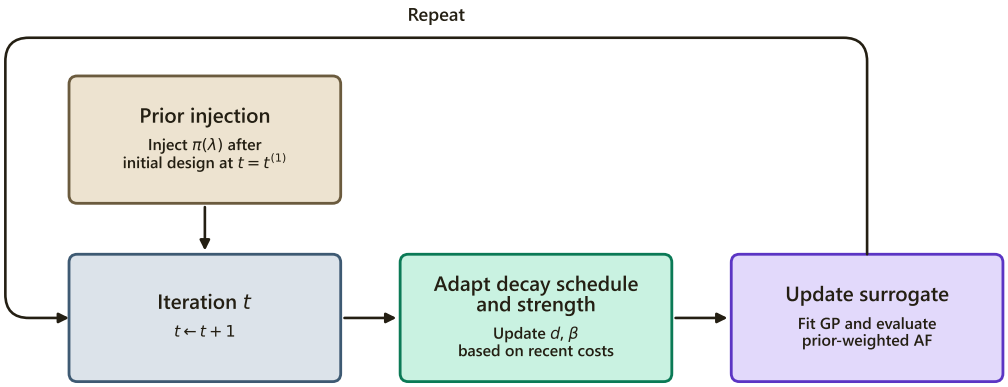


Figure 3.1.: Observation-adaptive decay for πBO: The prior $\pi(\lambda)$ is injected once after the initial design phase, after which every iteration t updates the decay schedule d and strength β from recent optimization costs before fitting the surrogate and evaluating the prior-weighted acquisition function.

3.1.1 Motivation

Before introducing observation-adaptive alternatives, we first examine how πBO's decay can be tweaked and what the implications are for different prior kinds. Throughout this thesis, Vanilla BO denotes standard BO without any prior, using the same surrogate model, acquisition function and initial design as all prior-guided variants.

Figure 3.2 plots mean incumbent regret on the y-axis against the number of evaluations on the x-axis, comparing Vanilla BO against four static πBO configurations that combine logarithmic or cubic decay with $\beta \in \{1.0, 10.0\}$, shown separately for an expert prior (left panel) and a deceptive prior (right panel) on `cifar100_wideresnet_2048`.

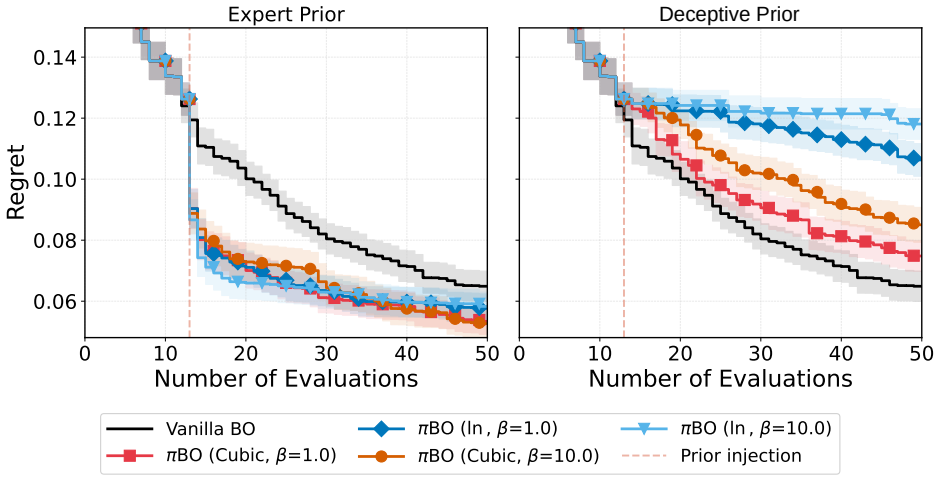


Figure 3.2.: Mean incumbent regret on `cifar100_wideresnet_2048` over 30 seeds for Vanilla BO and four static πBO configurations. The dashed vertical line marks the single prior injection after the initial design, shown for an expert prior (left) and a deceptive prior (right).

Given an expert prior, πBO outperforms Vanilla BO substantially regardless of decay schedule and strength. Despite both methods having a significant drop in regret after initial design, πBO's sharper drop to regret values smaller than 0.09 illustrates the benefit of prior-guided search. For $\beta = 10.0$, logarithmic decay outperforms cubic decay in the early iterations, since retaining the prior's numerical influence longer benefits an informative prior. Beyond iteration 35, however, cubic decay begins to outperform logarithmic, suggesting that the optimal decay for the prior term shifts as the surrogate accumulates evidence. This motivates adapting the schedule based on incoming observations rather than committing to a single fade-out rate in advance.

Under a deceptive prior, injecting the prior yields no regret reduction, but an increase. Moreover, the gap to Vanilla BO widens over time. The curves of the πBO configurations themselves split substantially: Both logarithmic decay variants see close to no reduction in regret, while the cubic schedules, particularly at $\beta = 1.0$, perform considerably better. They reduce the prior's influence on the equation more aggressively and thus converge faster toward Vanilla BO. Both scenarios confirm that multiple configurations can yield optimal results at different points in the optimization process. The optimal configuration depends jointly on prior quality and the current trial index, motivating observation-adaptive mechanisms that adjust the decay during optimization.

3.1.2 Generalized Decay Schedules

Recall from subsection 2.4.2 that πBO scales the acquisition function by a factor $\pi(\lambda)$, which we define as:

$$\alpha_{\pi,t}(\lambda, \mathcal{D}_t) \coloneqq \alpha(\lambda, \mathcal{D}_t) \cdot \pi_t(\lambda) = \alpha(\lambda, \mathcal{D}_t) \cdot \pi(\lambda)^{\gamma_t} \tag{3.1}$$

Since this factor stays the same, we want to decay its influence relative to the current iteration number t. That way the further into the optimization progress we are, the closer πBO's curve resembles Vanilla BO's. This is generally achieved by using an exponent γ_t that we take $\pi(\lambda)$ to the power of. To retain πBO's convergence guarantee (Hvarfner et al., 2022), this decay must satisfy

$$\lim_{t\to\infty} \pi_t(\lambda) = 1 \quad \text{and} \quad \lim_{t\to\infty} \gamma_t = 0. \tag{3.2}$$

While Hvarfner et al. (2022) uses $\pi_t(\lambda) = \pi(\lambda)^{\gamma_t} = \pi(\lambda)^{\beta/t}$ as the decay term, we explore different approaches for the general decay schedule. Here $t_{rel} \coloneqq t - t^{(m)} + 1$ counts the iterations elapsed since the m-th prior was injected at $t^{(m)}$ (following the injection notation of Fehring et al. (2026)), so that $t_{rel} = 1$ at the injection point itself. By modifying the term β is divided by in the denominator, we can achieve vastly different decay schedules for the exponent γ_t. Note that each schedule's name refers to how γ_t itself evolves over iterations, not to the shape of the resulting prior term $\pi_t(\lambda)$ directly. They each differ substantially in their impact on the prior's influence:

Table 3.1.: Decay schedules used for adaptive prior decay, parameterized by β and the elapsed iterations t_{rel}.

Decay schedule	**Definition**
`log10`	$\gamma_t = \beta / \log_{10}(t_{rel} + 1)$
`ln`	$\gamma_t = \beta / \ln(t_{rel} + 1)$
`linear`	$\gamma_t = \beta / t_{rel}$
`quadratic`	$\gamma_t = \beta / (t_{rel})^2$
`cubic`	$\gamma_t = \beta / (t_{rel})^3$

All of the above trivially satisfy Equation 3.2 and therefore inherit the convergence result of Hvarfner et al. (2022).

We want to compare the behavior of these schedules on a common scale. To that end, we report the *relative decay exponent* at iteration t as γ_t itself, relative to its undecayed value $\gamma_t = 1$, rather than the exponentiated term $\pi(\lambda)^{\gamma_t}$: The latter depends on λ and would therefore differ across configurations, whereas γ_t is the same for every λ and directly determines how strongly the prior term is weighted relative to the acquisition function at a given iteration, making it the natural configuration-independent quantity for comparing schedules.

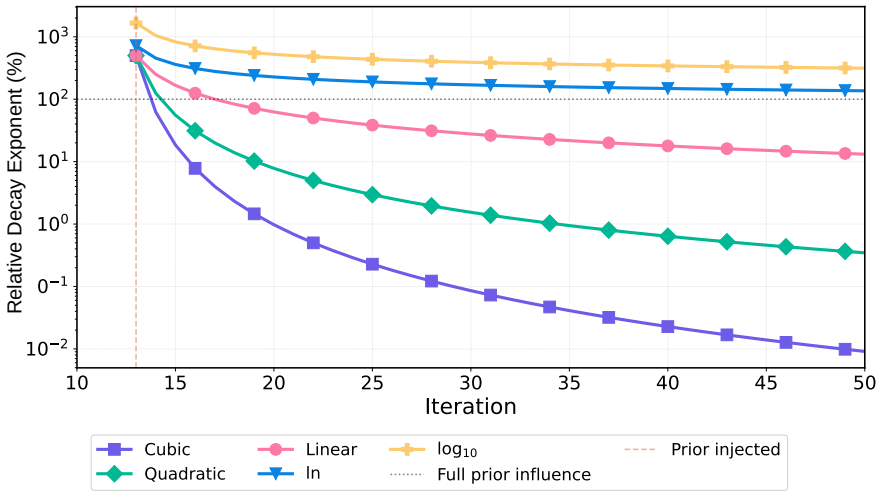


Figure 3.3.: Relative Prior Decay Exponent $\gamma_t \cdot 100\,\%$ over the course of a run for the five decay schedules, using $\beta = 5$.

The vast differences in the exponent depending on the used schedule are examined in Figure 3.3. At prior injection all graphs start out similarly, they do however separate substantially with increasing iterations of the algorithm. This is further illustrated in Table 3.2. Logarithmic decay schedules start out by increasing the prior's influence by a factor larger than β regardless of their respective base. Even after 50 iterations both logarithmic schedules still weigh the prior stronger than any constant heuristic that does not touch $\pi(\lambda)$. The other schedules however, do all start out with a value of 5 but then quickly decay. While linear decay does retain a decently large exponent even after 50 iterations, quadratic and especially cubic decay fade out the prior to essentially no impact.

Table 3.2.: Value of the exponent γ_t for each decay schedule at the injection point and at the end of the run, using $\beta = 5$. Post-injection iterations are indexed as $t^{(n)}$, so $t^{(1)} = 13$ denotes the injection point and $t = N = 50$ the final trial.

		Injection ($t = 13$)	Final trial ($t = 50$)
Schedule	γ_t	Value	Value
$\log_{10}$	$\beta / \log_{10}(t_{rel}+1)$	$5/0.301 = 16.61$	$5/1.591 = 3.14$
ln	$\beta / \ln(t_{rel}+1)$	$5/0.693 = 7.21$	$5/3.664 = 1.36$
Linear	β / t_{rel}	$5/1 = 5.00$	$5/38 = 0.13$
Quadratic	$\beta / (t_{rel})^2$	$5/1 = 5.00$	$5/1444 = 0.003$
Cubic	$\beta / (t_{rel})^3$	$5/1 = 5.00$	$5/54872 \approx 0$

No single schedule is therefore universally preferable: A logarithmic schedule is better suited to a prior that remains informative throughout the run, whereas quadratic or cubic decay is preferable once the prior's guidance is no longer needed and the surrogate should be allowed to dominate. This static trade-off is fixed for the entire optimization run once d and β are chosen, even though the point at which a prior stops being informative cannot be known in advance and varies across runs.

3.1.3 Oracle Decay

The oracle decay is constructed independently for each scenario and prior kind. For a given pair, every combination of decay schedule $d \in D$ and strength $\beta \in \mathcal{B}$ is evaluated by running πBO across S seeds and averaging the resulting incumbent loss curves. Let $\bar{\mathcal{L}}_t^{d,\beta}$ denote this seed-averaged incumbent cost at iteration t. In addition to the $|D| \cdot |\mathcal{B}|$ runs, we include the case $\varnothing$ of not injecting any prior at all, which is exactly Vanilla BO. Let $C \coloneqq (D \times \mathcal{B}) \cup \{\varnothing\}$ denote this set of $|D| \cdot |\mathcal{B}| + 1$ candidate configurations. The oracle cost is then defined as the pointwise minimum over C:

$$O_t = \min_{(d,\beta) \in C} \bar{\mathcal{L}}_t^{d,\beta} \tag{3.3}$$

The minimizing configuration may differ between iterations, so O_t forms a lower oracle bound over all averaged curves rather than the regret curve of any single run. Algorithm 2 illustrates the construction precisely.

Algorithm 2 Oracle Decay Construction

Require: Decay schedules D, exponent numerators $\mathcal{B}$, seeds S, trial budget N
for each $(d, \beta) \in (D \times \mathcal{B}) \cup \{\varnothing\}$ **do**
  for each seed $s \in \{1, \ldots, S\}$ **do**
    Run πBO with decay d, strength β, seed s
    (for $\varnothing$: run Vanilla BO, i.e. inject no prior)
    Record incumbent loss $\mathcal{L}_{t,s}$ for $t = 1, \ldots, N$
  end for
  $\bar{\mathcal{L}}_t^{d,\beta} \leftarrow \frac{1}{S} \sum_{s=1}^{S} \mathcal{L}_{t,s}$ ▷ Average over seeds
end for
for $t = 1, \ldots, N$ **do**
  $O_t \leftarrow \min_{(d,\beta) \in (D \times \mathcal{B}) \cup \{\varnothing\}} \bar{\mathcal{L}}_t^{d,\beta}$ ▷ Best configuration at each t
end for
return $\{O_t\}_{t=1}^{N}$

This construction is repeated independently for every scenario and prior kind, yielding a separate oracle curve $\{O_t\}_{t=1}^{N}$ for each.

3.1.4 Observation-Adaptive Decay

The oracle decay demonstrates that the optimal configuration (d, β) varies across iterations, yet the prior quality is unknown in advance. Analogous to Benjamins et al. (2023) this leads to the idea of adapting the decay schedule and its decay factor β online based on the observations collected during optimization.

Three variants are considered, each approaching the decay adaption from a different angle:

1. **Decay Schedule Adaptation:** The decay schedule d is selected from $\mathcal{D}$ at each iteration while β remains fixed.
2. **Decay Strength Adaptation:** The exponent numerator β is adjusted at each iteration while the schedule d remains fixed.
3. **Prioritized Decay Adaptation:** At each iteration, a priority rule decides whether d or β is adapted, so the two parameters are no longer restricted to being tuned independently.

For the sake of readability we later refer to them as *Adaptive Schedule*, *Adaptive* β and *Adaptive Schedule/*β respectively.

The family of heuristics use a running-cost signal derived from the optimization history. Let $\mathcal{L}_t$ denote the *observed* objective value at iteration t (a definition different from the actual incumbent). Let furthermore $h \coloneqq \max(1, \lfloor \mathcal{W}/2 \rfloor)$ split the window $\mathcal{W}$ into two contiguous halves. At iteration t we compute a newer-window average and an older-window average of the observed objective:

$$\bar{\mathcal{L}}_t^{\text{new}} \coloneqq \frac{1}{h} \sum_{i=t-h+1}^{t} \mathcal{L}_i, \qquad \bar{\mathcal{L}}_t^{\text{old}} \coloneqq \frac{1}{\mathcal{W}-h} \sum_{i=t-\mathcal{W}+1}^{t-h} \mathcal{L}_i. \tag{3.4}$$

We interpret $\bar{\mathcal{L}}_t^{\text{new}} > \bar{\mathcal{L}}_t^{\text{old}}$ as stagnation because the objective has not improved on average. This usage of half-windows reduces sensitivity to per-iteration noise in $\mathcal{L}_i$ compared to juxtaposing two single observations. When recent evaluations indicate progress we keep and even amplify the prior term's influence accordingly. Otherwise we decrease its impact more strictly than πBO's standard decay mechanisms do.

The heuristics that adapt the used decay schedule operate on a decay ladder we denote as $\mathcal{H}$, defined as the schedules of subsection 3.1.2 ordered from slowest to fastest fade-out:

$$\mathcal{H} = \big(\log_{10} \;\to\; \ln \;\to\; \text{Linear} \;\to\; \text{Quadratic} \;\to\; \text{Cubic}\big) \tag{3.5}$$

We write $\mathcal{H}^+$ for a single step toward faster fade-out and $\mathcal{H}^-$ for a single step toward slower fade-out, both clamped at the boundaries of $\mathcal{H}$, which we call $\mathcal{H}_{min}$ and $\mathcal{H}_{max}$. Applying $\mathcal{H}^+$ therefore reduces the prior's influence and $\mathcal{H}^-$ retains it longer. Note that $\mathcal{H}$ can be tuned as is necessary. Instead of stepping through discrete schedules, one could instead use a single polynomial schedule $\pi_t(\lambda) = \pi(\lambda)^{\beta/(t_{rel})^p}$ and let the heuristic increment or decrement p directly. The approach we take is more extreme and potentially leads to more interpretable results.

First Heuristic: Decay Schedule Adaptation

The first heuristic adjusts the decay schedule while keeping the prior strength β fixed. At each iteration we check whether the observation has on average improved in the last $\mathcal{W}$ evaluations. If the most recent observation is worse, we read this as stagnation and move one step up the ladder to fade the prior faster. Otherwise we move one step down and retain the prior longer:

$$(d, \beta)_{t+1} = \begin{cases} (\mathcal{H}^{+}(d_t),\, \beta) & \text{if } \bar{\mathcal{L}}_t^{\text{new}} > \bar{\mathcal{L}}_t^{\text{old}} \\ (\mathcal{H}^{-}(d_t),\, \beta) & \text{otherwise} \end{cases} \tag{3.6}$$

This formulation introduces $\mathcal{W}$ and the ladder composition $\mathcal{H}$ as new hyperparameters that can and should, in principle, be tuned per scenario.

Second Heuristic: Decay Strength Adaptation

The second heuristic keeps the decay function fixed and instead adapts the prior strength β multiplicatively. The same binary progress signal is used: When stagnating β is divided by a factor we call κ to accelerate the decay, and otherwise it is multiplied by κ to retain the prior term's influence:

$$(d, \beta)_{t+1} = \begin{cases} (d,\, \max(\beta_t/\kappa,\, \beta_{\text{min}})) & \text{if } \bar{\mathcal{L}}_t^{\text{new}} > \bar{\mathcal{L}}_t^{\text{old}} \\ (d,\, \min(\beta_t \cdot \kappa,\, \beta_{\text{max}})) & \text{otherwise} \end{cases} \tag{3.7}$$

The bounds $[\beta_{\text{min}}, \beta_{\text{max}}]$ aim to prevent extreme behavior and the concrete values used in our experiments are stated in subsection 4.2.2.

Third Heuristic: Prioritized Decay Adaptation

The third heuristic combines the mechanisms of the first two, but rather than adjusting both degrees of freedom at once, it prioritizes stepping along the decay ladder $\mathcal{H}$ and falls back to adapting the prior strength β only once the ladder has been exhausted in the required direction, utilizing $\mathcal{H}_{min}$ and $\mathcal{H}_{max}$:

$$(d, \beta)_{t+1} = \begin{cases} \big(d_t,\, \max(\beta_t/\kappa,\, \beta_{\text{min}})\big) & \text{if } \bar{\mathcal{L}}_t^{\text{new}} > \bar{\mathcal{L}}_t^{\text{old}},\ d_t = \mathcal{H}_{\text{max}} \\ \big(\mathcal{H}^{+}(d_t),\, \beta_t\big) & \text{if } \bar{\mathcal{L}}_t^{\text{new}} > \bar{\mathcal{L}}_t^{\text{old}},\ d_t \neq \mathcal{H}_{\text{max}} \\ \big(d_t,\, \min(\beta_t \cdot \kappa,\, \beta_{\text{max}})\big) & \text{if } \bar{\mathcal{L}}_t^{\text{new}} \leq \bar{\mathcal{L}}_t^{\text{old}},\ d_t = \mathcal{H}_{\text{min}} \\ \big(\mathcal{H}^{-}(d_t),\, \beta_t\big) & \text{if } \bar{\mathcal{L}}_t^{\text{new}} \leq \bar{\mathcal{L}}_t^{\text{old}},\ d_t \neq \mathcal{H}_{\text{min}} \end{cases} \tag{3.8}$$

Both branches push the two degrees of freedom in the same direction, so this variant can make more decisive adjustments at each step than either individual mechanism alone. It is however the most extreme heuristic, influencing the algorithm's trajectory the most.

3.2 Adaptive Prior Rejection for Multiple Injected Priors

This section develops the approach used to answer RQ2. In contrast to section 3.1 we examine the effect on multiple priors injected post initial design and will therefore make use of DynaBO and many of its proposed configurations (Fehring et al., 2026).

3.2.1 Fixed Decay Schedule

Instead of varying β or the decay schedule itself, we keep the decay fixed throughout all DynaBO experiments and use πBO's default linear decay schedule as outlined in subsection 2.4.2. This isolates the effect of online prior rejection we are trying to examine and is the main distinction between RQ2 and RQ3.

3.2.2 Oracle Rejection

We define the oracle rejection analogously to the oracle decay in subsection 3.1.3, but over two complementary strategy sets rather than a grid of decay parameters. Both sets enumerate acceptance and discard patterns directly and neither of them invokes DynaBO's criterion from Equation 2.9. That is why the rejection oracle is independent of τ and does not depend on its calibration.

Acceptance Combinations

Let $\mathcal{I}$ denote the set of prior injection points. The first strategy set $\mathcal{A}$ enumerates every possible accept/reject decision across these points:

$$\mathcal{A} = \{0, 1\}^{|\mathcal{I}|}, \quad |\mathcal{A}| = 2^{|\mathcal{I}|} \tag{3.9}$$

where 0 indicates a prior acceptance and 1 its rejection at each specific injection point $t \in \mathcal{I}$.

Random Prior Discards

The second strategy set $\mathcal{R}$ accepts every prior at injection but assigns each a random lifetime $\ell^{(i)} \sim \text{Uniform}(1, N - t^{(i)})$, discarding the prior's influence at iteration $t^{(i)} + \ell^{(i)}$. Since $\ell^{(i)}$

ranges over $\{1, \ldots, N - t^{(i)}\}$, the last discard iteration is N and every prior is discarded by the end of the run. For K independent draws:

$$\mathcal{R} = \left\{ r_k = (\ell_1^{(k)}, \ldots, \ell_{|\mathcal{I}|}^{(k)}) \;\middle|\; k = 1, \ldots, K \right\} \tag{3.10}$$

The two sets are not disjoint and thus the pattern of accepting all priors in $\mathcal{A}$ coincides with any draw in $\mathcal{R}$ that assigns every prior its maximum lifetime. Since the oracle is a pointwise minimum, this overlap is harmless however.

The oracle rejection is then the pointwise minimum over both strategy sets:

$$O_t = \min_{s \in \mathcal{A} \cup \mathcal{R}} \bar{\mathcal{L}}_t^s \tag{3.11}$$

where $\bar{\mathcal{L}}_t^s$ again denotes the seed-averaged incumbent cost of strategy s at iteration t. Both strategy sets are constructed independently, as detailed in Algorithm 3 and Algorithm 4.

Algorithm 3 Acceptance-Pattern Averaging

Require: injection points $\mathcal{I}$, seeds $\{1, \ldots, S\}$, trial budget N
Ensure: seed-averaged incumbent costs $\bar{\mathcal{L}}_t^a$ for all $a \in \mathcal{A} = \{0, 1\}^{|\mathcal{I}|}$

for each $a \in \mathcal{A}$ **do**
  for each seed $s \in \{1, \ldots, S\}$ **do**
    Run DynaBO with rejection pattern a, seed s
    Record incumbent loss $\mathcal{L}_{t,s}^a$ for $t = 1, \ldots, N$
  end for
  $\bar{\mathcal{L}}_t^a \leftarrow \frac{1}{S} \sum_{s=1}^{S} \mathcal{L}_{t,s}^a$
end for

Algorithm 4 Random-Discard Averaging

Require: injection points $\mathcal{I}$, seeds $\{1, \ldots, S\}$, trial budget N, draw count K
Ensure: seed-averaged incumbent costs $\bar{\mathcal{L}}_t^{r_k}$ for $k = 1, \ldots, K$

for $k = 1, \ldots, K$ **do**
  Sample lifetimes $r_k = (\ell_1, \ldots, \ell_{|\mathcal{I}|})$ with $\ell_i \sim \text{Uniform}(1, N - t_i)$
  for each seed $s \in \{1, \ldots, S\}$ **do**
    Run DynaBO accepting all priors, discarding prior i after ℓ_i iterations
    Record incumbent loss $\mathcal{L}_{t,s}^{r_k}$ for $t = 1, \ldots, N$
  end for
  $\bar{\mathcal{L}}_t^{r_k} \leftarrow \frac{1}{S} \sum_{s=1}^{S} \mathcal{L}_{t,s}^{r_k}$
end for

Note how unlike subsection 3.1.3 we do not include Vanilla BO in the rejection oracle. Since Algorithm 3 includes the case of rejecting every prior ($a = (1, 1, 1, 1)$) we need not include Vanilla BO, as its trajectory would be equal to that of DynaBO using the reject-all configuration in $\mathcal{A}$.

3.2.3 Per-Step Reassessment of Accepted Priors

We want to investigate whether we can narrow the gap between the basic DynaBO approach proposed by Fehring et al. (2026) and our oracle defined in subsection 3.2.2 by changing the way priors are validated. In the current implementation by Fehring et al. (2026) priors are either accepted or discarded on injection. We will explore the idea of re-evaluating this on a stepwise basis.

Stepwise Prior Rejection Check

In standard DynaBO, the rejection criterion from Equation 2.9 is evaluated once at injection time and the decision persists for the entire run. The stepwise prior rejection check modifies this by re-evaluating the same criterion at every subsequent iteration using the current surrogate $\hat{f}_t$. Note that, while this implies the possible re-inclusion of discarded priors, we purposefully limit prior rejections to being final. In practice, this means that if the LCB difference falls below τ at any point, the prior is dropped. This allows the optimizer to revoke an initially accepted prior once the surrogate has gathered enough evidence that the prior region is no longer promising.

We are effectively doing the following for every non-discarded prior at every iteration:

$$\pi_{t+1}(\lambda) = \begin{cases} \pi_t(\lambda) & \text{if } \mathbb{E}_{\lambda \sim F_{\pi_t}}\left[\xi_{\hat{f}_t}(\lambda)\right] - \mathbb{E}_{\lambda \sim \mathcal{N}_{\hat{\lambda}}}\left[\xi_{\hat{f}_t}(\lambda)\right] \geq \tau, \\ 1 & \text{otherwise.} \end{cases} \tag{3.12}$$

Once the otherwise branch is taken, the prior is permanently added to the set of discarded priors and is excluded from all subsequent reassessments, so $\pi(\lambda) = 1$ for the remainder of the run.

Adapting the stepwise Prior Rejection Check

Moving from a single check at injection to the stepwise re-evaluation introduced above changes what the threshold τ has to guard against. Under DynaBO, τ has to be strict enough that a misleading prior is never accepted, because admission at injection is final. Once the criterion is re-evaluated at every iteration, a prior wrongly admitted at injection can still be caught a few evaluations later. We therefore postulate that τ does not need to be as strict under the stepwise check. While Fehring et al. (2026) suggest a value of $\tau = -0.15$ for the injection-only check, we will try to loosen this threshold for the stepwise variant. A looser threshold should usually benefit well-suited priors, since it falsely rejects fewer of them. It is however only reasonable as long as it does not have a significant negative impact on scenarios using deceptive prior choices.

3.3 Combined Mechanism

Finally, we combine these two ideas. First, subsection 3.1.4 adapts the decay schedule and its parameters based on current observations, while subsection 3.2.3 discards priors outright at every iteration step. The two mechanisms target different failure modes and can be applied in conjunction.

Concretely, the combined mechanism applies the joint decay adaptation of Equation 3.8 on top of the stepwise rejection check of Equation 3.12. We adapt the parameters *per prior*: Each injected prior carries its own (d_t, β_t) state, initialized at (d_0, β_0) at its own injection point, and its window is measured relative to that point rather than to the start of the run. This keeps the mechanism identical to the πBO case for the first prior and avoids a newly injected prior inheriting state from an earlier candidate.

Having motivated and specified the adaptive decay schedule, the stepwise rejection check, and their combination, we now turn to evaluating all three empirically, as chapter 4 describes the experimental setup shared across all approaches.

4 Experiments

This chapter describes the experimental protocol used to address the three research questions posed in chapter 1: After outlining the experimental setup in section 4.1, section 4.2 investigates observation-adaptive decay schedules utilizing πBO, section 4.3 then addresses adaptive rejection criteria for DynaBO, while section 4.4 evaluates their combination.

4.1 Experimental Setup

We evaluate all methods on the PD1 benchmark (Wang et al., 2024), a large-scale dataset of optimizer hyperparameter evaluations collected by training thousands of deep learning models across image and language tasks. We access the surrogate-predicted evaluations through mfpbench (Mallik et al., 2023), which is accessed through the CARP-S benchmarking suite (Benjamins et al., 2025). Following Fehring et al. (2026), we use the four scenarios listed in Table 4.1 to evaluate our results.

Table 4.1.: PD1 benchmark scenarios (Wang et al., 2024).

Alias	Model	Dataset
`widernet`	Wide ResNet	CIFAR-100
`resnet`	ResNet-50	ImageNet
`transf`	Transformer	LM1B
`xformer`	Transformer	WMT15

We measure performance by the simple regret $R_t = \mathcal{L}(\hat{\lambda}_t) - \mathcal{L}(\lambda^*)$ after a budget of t evaluations, where $\hat{\lambda}_t$ is the incumbent, i.e. the best configuration observed so far, and λ^* denotes the global optimum of the surrogate. Lower regret therefore indicates that the optimizer has found a configuration closer to the surrogate-optimal one.

Following the data-driven construction protocol of Fehring et al. (2026), all priors are represented as Gaussian distributions centered on a configuration λ^+ selected from a set of pre-clustered configurations. They are obtained by conducting an extensive exploratory search to gather a large pool of configuration-loss pairs $(\lambda, f(\lambda))$. These are then partitioned into n clusters $\{c_1, c_2, \ldots, c_n\}$ via agglomerative clustering using Gower's distance (Gower, 1971) and Ward linkage. Gower's distance is chosen since it naturally handles the mixed continuous and categorical hyperparameters present in the PD1 search spaces. Each cluster c_i is summarized by its centroid and its median loss l_{c_i}, and the clusters are indexed by increasing median loss, $l_{c_i} \leq l_{c_j}$ for $i < j$. That way, a higher rank i corresponds to a worse-performing region of the configuration space. The prior kinds below differ only in

how they draw a cluster c^+ from this ranking, conditioned on the current incumbent $\hat{\lambda}$ and its loss $l_{\hat{\lambda}}$, and additionally how they place the prior center λ^+ within it.

Analogous to Fehring et al. (2026), we evaluate prior-guided BO under different prior quality conditions that span the realistic range of user-provided beliefs.

Expert priors direct the search toward configurations that strictly outperform the current incumbent. A cluster c^+ is drawn from these qualifying clusters with probability $P(c_i) \propto e^{0.1 \cdot i}$, and the prior center is set to the best configuration within that cluster, $\lambda^+ \in \arg\min_{\lambda \in c^+} l_\lambda$.

Advanced priors are constructed analogously but relax the center selection: The cluster is drawn with $P(c_i) \propto e^{0.15 \cdot i}$, and the prior center $\lambda^+ \in c^+$ is chosen uniformly at random rather than greedily.

Local priors exploit knowledge of the current incumbent's position rather than global loss rankings: the ten clusters closest to $\hat{\lambda}$ under Gower's distance (Gower, 1971) are retained, the cluster with the lowest median loss is selected, and a configuration is sampled uniformly at random as the prior center. This setting models a practitioner who reasons about performance locally, a form of prior knowledge qualitatively different from the global cluster-ranking strategies above. We include this kind only to motivate the variant below and do not evaluate it further, since its near-greedy selection makes it behave much like the expert and advanced kinds.

Local Random priors modify the local priors of Fehring et al. (2026). To better represent our experimental evaluations, we sample from a larger neighborhood of clusters, going from the original 10 to 25. For the PD1 benchmark used predominantly in this thesis, this is equivalent to selecting one quarter of the available clusters, since we cap the cluster count at 100 for each scenario. Then, instead of selecting the cluster with the lowest median loss, we sample one of the 25 clusters uniformly at random. This adds reasonable randomness to the approach without relying on a purely random selection of priors.

Deceptive priors simulate a worst-case scenario in which the user's belief is actively misleading: the prior center is set to $\lambda^+ \in \arg\max_{\lambda \in c^+} l_\lambda$, drawn from one of the five clusters with the highest median loss, thereby biasing the search toward poorly performing regions of the configuration space.

4.1.1 Shared Experimental Configuration

To ensure comparability, all experiments share the configuration summarized in Table 4.2. Throughout the following chapters, Vanilla BO denotes standard BO under this same configuration with no prior injected, using Expected Improvement directly without any prior-induced reweighting of the acquisition function, with candidate configurations proposed via SMAC's combined local and random search maximizer, instantiated with 5000 random challengers and a local search capped at 500 steps.

Table 4.2.: Shared experimental configuration used across all PD1 experiments in this chapter.

Component	Value
Surrogate model	Random Forest
Acquisition function	Expected Improvement
Trial budget N	50
Initial design	12 Sobol configurations
Seeds	30
Prior injection $t^{(1)}$	13

4.1.2 Statistical Testing Protocol

Because the differences between variants are frequently small relative to the seed-to-seed spread, we support the aggregate comparisons in chapter 5 with paired Wilcoxon signed-rank (Wilcoxon, 1945) tests computed separately for each scenario, pairing a variant and its baseline on matched per-seed runs. The four resulting one-sided p-values are combined across scenarios via Stouffer's method (Stouffer et al., 1949), weighted by the number of matched seed pairs per scenario, which is uniform here and therefore reduces to an unweighted combination. We test the directional hypothesis that a variant improves on its baseline, so the reported markers indicate significant improvement only. We report significance at three levels ($p < 0.01$, $p < 0.05$, $p < 0.1$) and otherwise describe the ordering of variants as directional.

4.1.3 Area over the Curve and Gap Closed

We use area under the curve (AUC) as the simple integral of our regret trajectories and we similarly define the area over the curve (AOC) as the area between a method's incumbent curve and the oracle as our main aggregate metric (López-Ibáñez et al., 2014). It summarizes anytime performance in a single number, whereas final regret captures only the terminal iteration. Concretely, for a method m with mean incumbent regret curve $\mathcal{L}_m(t)$ and oracle curve $\mathcal{L}_{\text{oracle}}(t)$ defined in subsection 3.1.3, AOC is the trapezoidally integrated gap between the two curves over the run,

$$\text{AOC}_m = \frac{1}{N-1} \int_1^N \left(\mathcal{L}_m(t) - \mathcal{L}_{\text{oracle}}(t) \right) dt, \tag{4.1}$$

so that a smaller AOC means a method tracks the oracle more closely across the whole budget, and $\text{AOC}_{\text{oracle}} = 0$ by construction.

To express how much of the gap between static linear πBO and the oracle a variant closes, we report the *gap closed*,

$$\text{Gap Closed}(m) = 100 \cdot \frac{\text{AOC}_{\text{linear}} - \text{AOC}_m}{\text{AOC}_{\text{linear}}}, \tag{4.2}$$

where a value of $100\,\%$ indicates that m fully closes the gap to the oracle, $0\,\%$ indicates no improvement over static linear πBO and DynaBO respectively, and negative values indicate that m performs worse than the static linear variants.

4.2 RQ1: Adaptive Decay for πBO

To address RQ1, we compare πBO's fixed decay schedule against the oracle decay of Equation 3.3 and against three observation-adaptive heuristics, whose experimental details are outlined in the following section.

4.2.1 Oracle Prior Decay

The oracle decay is defined as outlined in Equation 3.3, with $\mathcal{B} = \{1.0, 2.0, \ldots, 20.0\}$, yielding $|\mathcal{B}| = 20$ values and $\mathcal{D} = \{\log_{10}, \log_e, \text{linear}, \text{quadratic}, \text{cubic}\}$. Each point of O_t is therefore the minimum over $|\mathcal{B}| \times |\mathcal{D}| + 1 = 101$ cost values (the $+1$ being the no-prior Vanilla BO case of Equation 3.3), each of which has itself been averaged over 30 seeds.

4.2.2 Observation-Adaptive Decay Heuristics

All three adaptive-decay heuristics of subsection 3.1.4 share the same configuration. The scaling factor is generally set to $\kappa = 1.5$ throughout this thesis, the strength bounds to $[\beta_{\text{min}}, \beta_{\text{max}}] = [1, 50]$, and the window size to $\mathcal{W} = 4$. Each run is initialized at the injection point with the linear schedule and the fixed strength used for static linear πBO, i.e. $d_0 =$ linear and $\beta_0 = 10$, and adapts from there along the ladder $\mathcal{H}$ of Equation 3.5. For πBO a single prior is injected at $t^{(1)} = 13$, so the decay adaptation acts over iterations $t \in [13, 50]$ only. For Equation 3.8 we start the observation adaptivity two iterations later to circumvent high regret values in the first iterations after prior injection. Additionally, for this heuristic we set $\kappa = 2.0$, since it is not expected to be applied at every iteration but only when we are already at the decay ladder's bounds. The window size $\mathcal{W}$ is the one hyperparameter we vary deliberately: A sensitivity analysis sweeping $\mathcal{W} \in \{1, 2, \ldots, 20\}$ is reported in the following chapter to justify the choice of $\mathcal{W} = 4$.

4.3 RQ2: Adaptive Rejection for DynaBO

To address RQ2, we analogously compare DynaBO's fixed rejection against the oracle rejection of Equation 3.11 and against heuristics utilizing stepwise prior inspections. We discuss the experimental details in the following parts:

4.3.1 Oracle Prior Rejection

The oracle rejection is defined as outlined in Equation 3.11 with $|\mathcal{I}| = 4$, since we inject a prior every 10 iterations after the initial design ($t \in \{13, 23, 33, 43\}$). The acceptance-combination set $\mathcal{A}$ therefore contributes $2^4 = 16$ patterns. To approximate the random-discard set $\mathcal{R}$ we additionally sample $K = 100$ lifetime draws, each accepting every prior at injection and assigning it a random lifetime as shown in subsubsection 3.2.2. Thus, we effectively sample a range of possibilities for what could be gained by discarding priors mid-run. In total the oracle is the pointwise minimum over $16 + 100 = 116$ candidate strategies, each of which has itself been averaged over 30 seeds.

4.3.2 Shared DynaBO Configuration

To isolate the effect of prior rejection from that of adaptive decay, all DynaBO experiments use πBO's linear decay schedule with a fixed β, which we calibrate once in a preliminary sweep and then hold constant for the rejection experiments in this thesis. Table 4.3 previews the two values of practical interest, with the full sweep underlying this choice reported in Table A.1 and Table A.2.

Table 4.3.: AOC and final regret at $\beta = 5$ (the literature recommendation) versus $\beta = 10$ (adopted here), aggregated over all prior kinds (30 seeds, 4 scenarios, PD1). For both smaller is better. For full results see Table A.1 and Table A.2

	AOC ($\times 10^{-2}$)	**Final Regret**
$\beta = 5$	1.128 ± 0.088	0.052 ± 0.001
$\beta = 10$	1.079 ± 0.088	0.051 ± 0.001

Both Hvarfner et al. (2022) and Fehring et al. (2026) recommend $\beta = N/10$, which for our budget of $N = 50$ yields $\beta = 5$. Table 4.3 shows that doubling this to $\beta = 10$ costs essentially nothing in AOC or final regret while letting less trustworthy priors influence the search for longer. A full sweep confirms both metrics are largely insensitive to β. To highlight the effects of our proposed mechanisms, we choose $\beta = 10$ for our rejection experiments, as it is not the empirical optimum but a solid middle of this plateau, retaining prior influence longer than the standard recommendation without sitting at any extreme.

4.4 RQ3: Combined Mechanism

To address RQ3, we compare and combine the findings of the two previous RQs and evaluate them in conjunction.

4.4.1 Combined Oracle

The combined approach does not introduce a separate oracle. Because constructing a joint oracle is computationally infeasible, we report against the rejection oracle, which is therefore a reference rather than a strict upper bound.

4.4.2 Combining Rejection Heuristics

As specified in section 3.3, the combined approach augments the DynaBO experiments with the prioritized decay adaptation of subsection 4.2.2: In addition to the stepwise rejection check, the decay schedule and its strength are adapted per prior based on the observations gathered so far. All shared hyperparameters (κ, $\mathcal{W}$, the strength bounds, and the initial state) take the same values as previously outlined.

5 Results

This chapter presents and discusses the experimental results obtained for each research question, structured to mirror the two mechanisms introduced in chapter 3. All results are reported on the PD1 benchmark under the experimental protocol described in section 4.1, and the comparisons are supported by the Wilcoxon signed-rank test. Then, section 5.1 addresses observation-adaptive prior decay, building on the static decay schedule of Hvarfner et al. (2022), opening with the prior decay oracle in order to establish which combinations of decay schedule and prior strength β are optimal for each prior kind, an analysis that motivates the three heuristics evaluated afterwards: Adapting the decay schedule alone, adapting the prior strength β alone, and combining both. The section closes with a sensitivity analysis of the sliding window size $\mathcal{W}$ that triggers the adaptive mechanisms, followed by a joint numerical comparison of all decay variants against the static linear baseline. Observation-adaptive prior rejection is covered in section 5.2, extending the injection-time acceptance criterion of Fehring et al. (2026), which similarly opens with the prior rejection oracle to isolate how much of its headroom stems from injection-time decisions versus retroactive discards, motivating a stepwise rejection check evaluated at every iteration rather than only at injection. We then examine the sensitivity of the rejection threshold τ to this stepwise formulation, apply the resulting less strict threshold as a second heuristic, and analyze how often priors are discarded under both thresholds. The adaptive decay and rejection mechanisms are finally evaluated in conjunction in section 5.3, which compares the best-performing adaptive variants of both against their respective baselines and oracles, contrasts stepwise and injection-only rejection directly, and closes with a joint numerical evaluation of all adaptive rejection variants.

Shared Conventions for Incumbent Regret Plots Unless stated otherwise, the following step function plots show the four prior kinds introduced in section 4.1. The x-axis denotes the number of evaluations, and dashed vertical lines mark the points of prior injection. Each curve is the mean incumbent regret, averaged over 30 seeds for PD1, with shaded bands showing the corresponding standard error. Where included, the grey oracle curve is the pointwise minimum over all candidate configurations defined in subsection 3.1.3 for decay plots or subsection 3.2.2 for rejection plots, and serves as a lower reference rather than an attainable method.

5.1 Observation-Adaptive Prior Decay

First, we examine prior decay in isolation, using the simple regret metric introduced in section 4.1, injecting a single prior of varying quality once ($t^{(1)} = 13$) after initial design and evaluating our proposed dynamic decay approaches.

5.1.1 Analyzing the Prior Decay Improvement Potential

Figure 5.1 examines the effect of β and the decay schedule d in the exponent on anytime regret. For a decay configuration (d, β) we define its excess over the oracle as the relative gap between the area under its seed-averaged incumbent regret curve and the area under the oracle decay of subsection 3.1.3, so that $0\,\%$ marks a configuration that matches the oracle and larger values are worse, mirroring the AOC-based gap closed metric introduced in section 4.1.

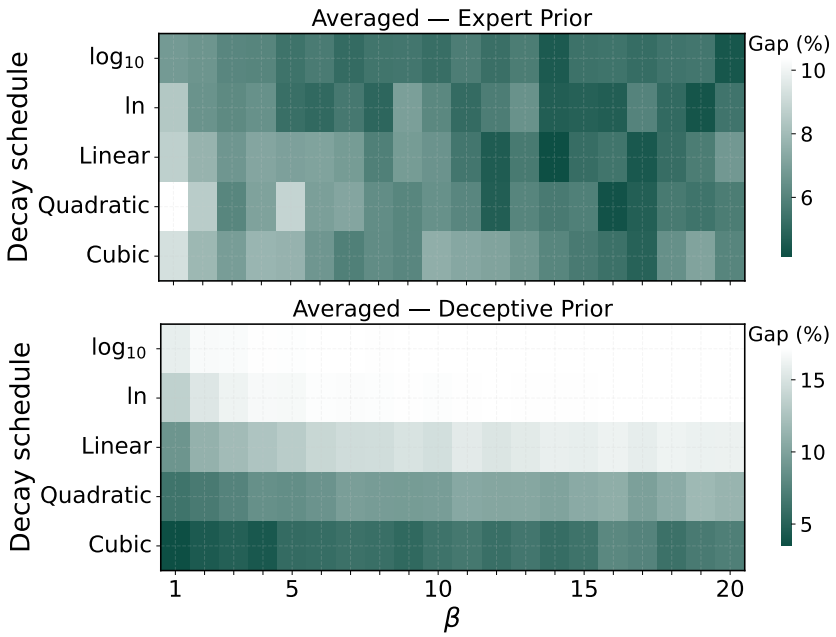


Figure 5.1.: Excess over the oracle decay for every combination of decay schedule and prior strength β, averaged over all four PD1 scenarios, for expert priors (top) and deceptive priors (bottom). Each cell reports the relative gap between the area under the corresponding incumbent regret curve and that of the oracle decay defined in subsection 3.1.3. Lower values are better.

We distinguish by prior kinds to better analyze the optimal configurations for each. It is apparent that the heatmaps vary substantially depending on the used prior kind. While **deceptive** priors greatly benefit from cubic and quadratic decays with low values for β, **expert** priors benefit from the opposite corner of the grid, namely the slow logarithmic schedules using higher β. For full results see Figure A.1.

The plots do confirm what we already expected: Depending on the prior kind we should utilize different strategies to best optimize for low regret. Intuitively this makes sense, as well-tuned priors are by definition more useful and should thus, in theory, be retained for longer periods of time. Actively harmful priors on the other hand should be faded out as quickly as possible, as they lead the optimization process into less useful regions. That is why the deceptive prior heatmap is so skewed towards cubic decay with lower values for β. This finding, in conjunction with the reliance on the relative iteration we outlined in subsection 3.1.1, directly leads into the observation-adaptive approaches discussed in the following section.

5.1.2 Adaptive Decay Schedule (First Heuristic)

We first evaluate the heuristic of subsubsection 3.1.4, which adapts the decay schedule d alone while keeping β fixed, against vanilla πBO and Vanilla BO. Figure 5.2 shows the resulting incumbent regret on `cifar100_wideresnet_2048` across the four prior kinds. The heuristic lowers regret most clearly where the injected prior is least reliable, while leaving the informative cases essentially unchanged.

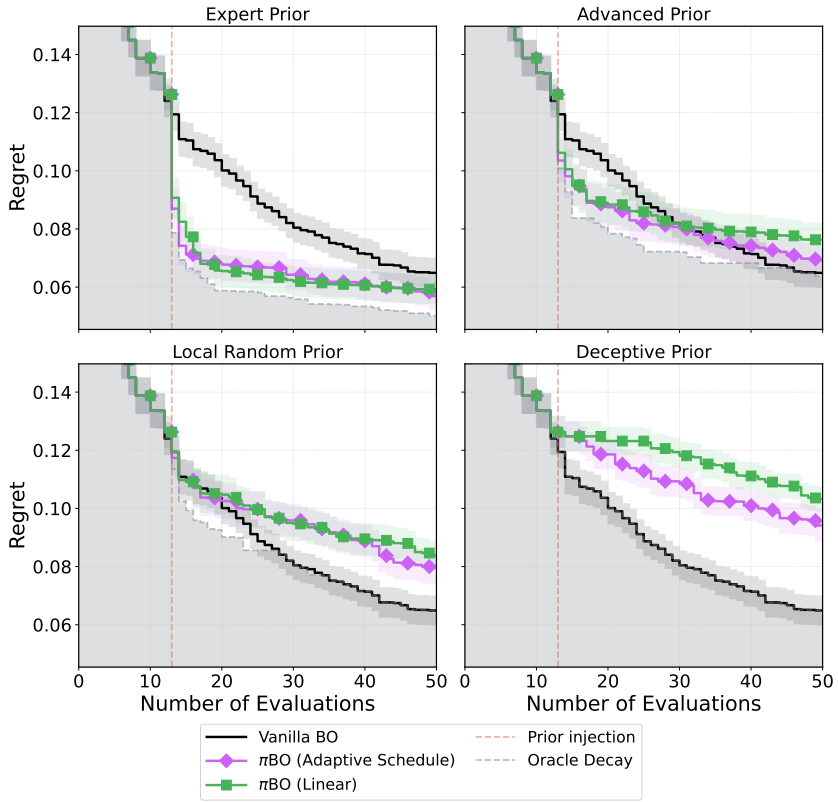


Figure 5.2.: Mean incumbent regret per prior kind for standard linear πBO, adaptive-schedule πBO (subsubsection 3.1.4), and Vanilla BO on `cifar100_wideresnet_2048` using $\mathcal{W} = 4$, with the oracle decay of subsection 3.1.3 as a lower reference and bands giving the standard error over 30 seeds. All PD1 scenarios are shown in Figure A.2.

Generally we see that all trajectories before prior injection at iteration 13 are the same. The parts we analyze and adapt therefore only contain iterations $t \in [13, 50]$. All curves, excluding the one displaying the **deceptive** and to some extent the **local random** prior, show similar behaviour directly after prior injection. There is a steep drop in regret directly after initial design making πBO perform better than Vanilla BO. However, the drop is steeper the better the prior kind is. That is why for the **expert** prior regret drops almost to oracle decay levels at first while regret for the other prior kinds stays higher.

The **expert** prior panel does not show significant difference between our heuristic and basic πBO. Both clearly outperform Vanilla BO but neither shows any substantial difference in anytime or final regret. However, their values are already very close to the oracle regret we established and thus no large reductions were expected.

For the **advanced** prior we can see slightly lower regret using our heuristic. The gap furthermore gets larger with more iterations. Note that Vanilla BO still outperforms both πBO using vanilla configurations and our heuristic towards later iterations though. Vanilla BO coincides with the oracle towards the end of the run, as the oracle decay of subsection 3.1.3 includes the no-prior case as a candidate. That is why wherever injecting no prior is the best available strategy the oracle's trajectory is equal to Vanilla BO's.

Vanilla BO behaves similarly for the **local random** prior kind: It starts off having similar if not slightly higher regret than the two πBO configurations but starts to outperform them with increasingly large t. However, the **local random** demonstrates the decrease in regret our approach can offer. There is a noticeably large gap between πBO using the linear decay schedule and our observation-adaptive approach towards the final few iterations. For the **local random** and **deceptive** prior kinds the oracle decay's regret is the lowest relative to πBO's course, suggesting the largest room for improvement to be found in cases where the injected priors are less useful.

Deceptive priors demonstrate this the best. We observe a significant gap to πBO when using our heuristic while simultaneously closing the gap to the oracle decay. The adaptive schedule still clearly gets outperformed by simply injecting no prior at all, again suggesting that the best course of action would just be to use Vanilla BO when prior quality is questionable. However, the large reduction in regret confirms that we can in fact narrow the gap to the oracle decay in a meaningful way for some scenarios when adapting the exponent's decay schedule online.

5.1.3 Adaptive Prior Strength (Second Heuristic)

Adapting the prior strength β alone (subsubsection 3.1.4) does not yield any promising results on any of the used benchmarks, as Figure 5.3 shows across the four prior kinds.

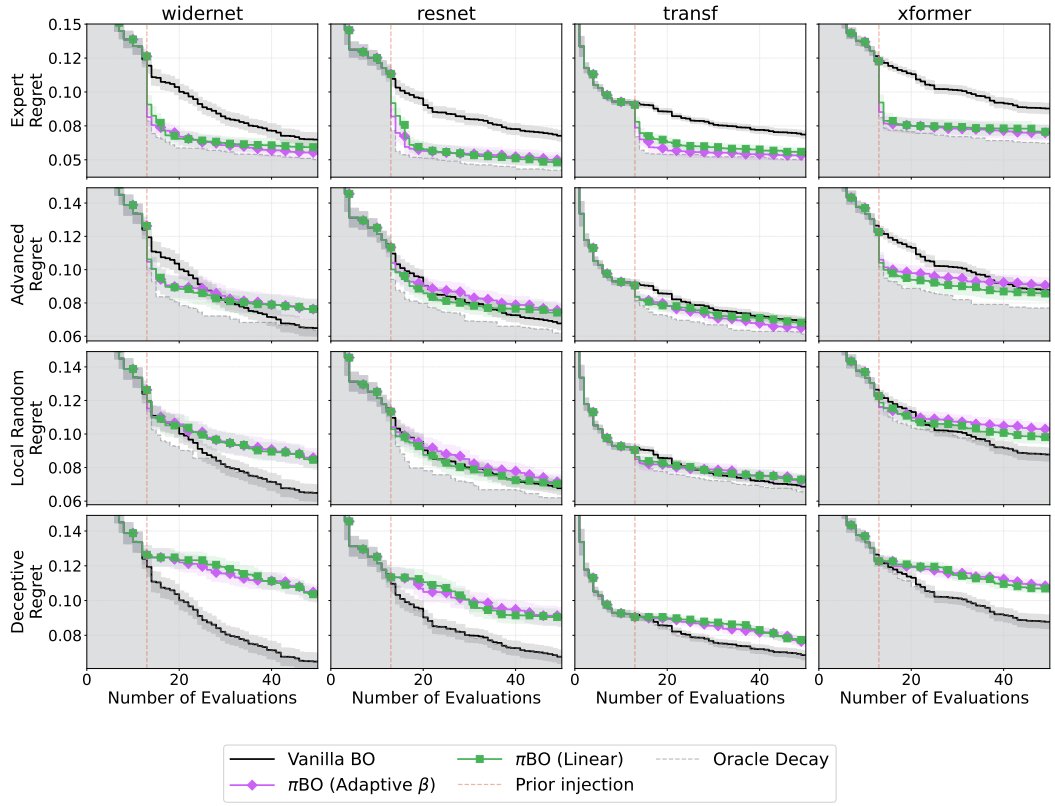


Figure 5.3.: Mean regret per prior kind for basic vanilla πBO and adaptive-strength πBO (subsubsection 3.1.4) on all PD1 scenarios using $\mathcal{W} = 4$. The per scenario average is shown in Figure A.5.

While some prior kinds do perform slightly better using our observation adaptive approach, others do not. Note however, that the approach does not perform substantially worse than πBO using Hvarfner et al. (2022)'s configurations. The two approaches on average behave more or less the same with marginal differences in some cases skewing the regret in both possible directions. But one could conclude that only changing β mid optimization does not yield any promising results with expert priors being the only small exception, although its narrowed gap to the oracle cannot be called significant, as we demonstrate later.

For this specific decay mechanism, Figure 5.4 illustrates the resulting decay behavior for a single sample run on `cifar100_wideresnet_2048`, tracking β and the prior decay exponent separately for each prior kind after injection. We define the latter as the current decay exponent γ_t, relative to the exponent at the first iteration after injection under the initial linear schedule with β_0, expressed as a percentage. Concretely, this means that values above $100\,\%$ indicate the heuristic has increased the prior's impact beyond its starting strength, while values below $100\,\%$ indicate a fading prior. This notation helps us track the prior numerically without relying on the actual values of $\pi_t(\lambda)$ at all.

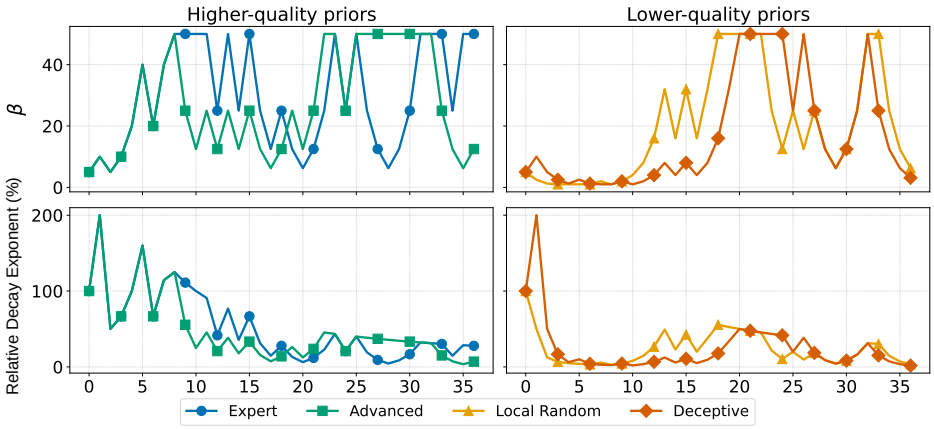


Figure 5.4.: Post-injection prior impact and value for β for a single sample run on `cifar100_wideresnet_2048` utilizing the adaptive decay heuristic defined in subsubsection 3.1.4.

The overall picture is clear: The prior decay exponent decays over time regardless of prior kind. However, in early iterations following prior injection especially, its influence is way lower for the lower quality priors. While prior influence stays above 50% for almost half of the iterations for expert and advanced priors, it almost immediately drops to a few percent points for deceptive and local random priors. We can see that this behavior stems from β being divided repeatedly for the worse priors and being magnified in the other panel. But even deceptive and local random priors do eventually see a big increase in their respective decay numerators. However at this point (iteration 15 onwards) the resulting gains in the prior term's influence on the equation are almost negligible.

The charts demonstrate that the mechanism works as intended, we do not get stuck at the respective bounds and the better the prior kind, the longer the heuristic retains or even amplifies its influence. Early spikes as seen in the deceptive prior's trajectory for example can be attributed to random noise and are furthermore promptly counteracted by the same mechanisms that led to them in the first place. An analogous chart for Equation 3.6 can be found in Figure A.3

5.1.4 Combined Adaptive Decay and Strength (Third Heuristic)

Combining the online adaptation of β with the online adaptation of the decay schedule yields the largest gains of the three heuristics whenever the injected prior is unreliable, an effect clearly visible for the **deceptive** prior kind, not only on `cifar100_wideresnet_2048` (Figure 5.5) but across all PD1 scenarios.

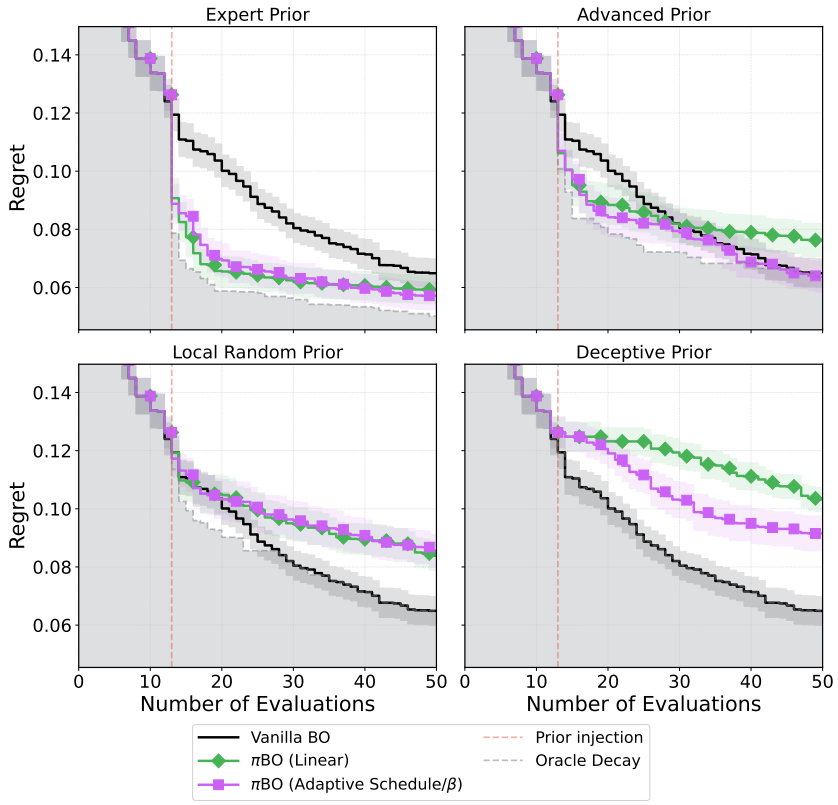


Figure 5.5.: Mean regret per prior kind for standard linear πBO and combined adaptive πBO (subsubsection 3.1.4) on `cifar100_wideresnet_2048` using $\mathcal{W} = 4$. All PD1 scenarios with the oracle are shown in Figure A.6.

On `cifar100_wideresnet_2048` specifically, the **expert** and **local random** priors do not show any significant regret reduction for our combined heuristic, whereas the **advanced** and **deceptive** prior panels show a much larger effect. Combining the two equations evaluated earlier yields the largest gains when the injected prior is least reliable, and clearly outperforms πBO using default configurations. While the difference is small in the beginning right after initial design, the gap appears to widen with more iterations, especially noticeably in the **advanced** prior panel on `cifar100_wideresnet_2048`. Notice again that even where our heuristic does not reduce regret, it does not increase it either. Instead we observe similar incumbent curves to πBO or outperform it in nearly every scenario and prior combination.

5.1.5 Sensitivity Analysis of the Window Size $\mathcal{W}$ Introduced by Selected Adaptive Heuristics

Having established how the schedule and its hyperparameters are adapted during optimization, we now examine the one hyperparameter of the trigger itself, the sliding window that checks whether on average costs have improved recently. Figure 5.6 plots the influence of $\mathcal{W}$ on the gap closed relative to standard πBO for the three adaptive decay heuristics.

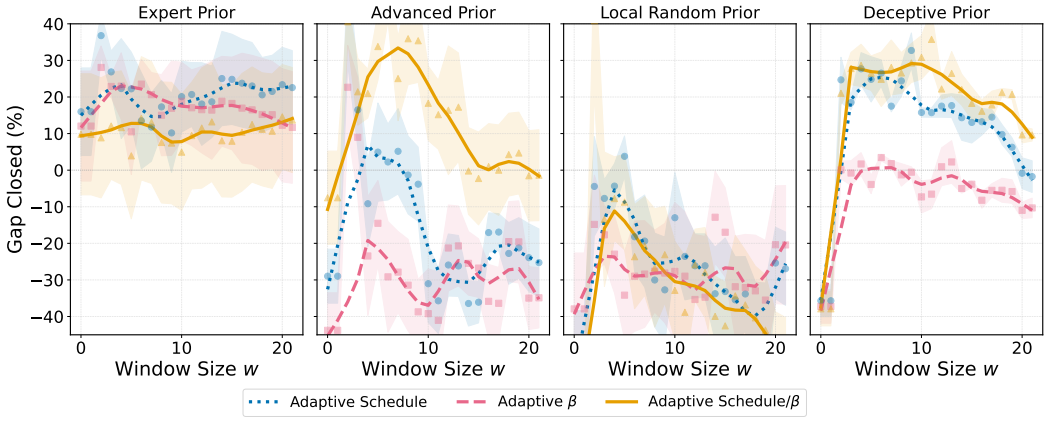


Figure 5.6.: Gap closed relative to πBO with static linear decay as a function of the window size $\mathcal{W}$, per prior kind, for our adaptive decay mechanisms defined in subsection 3.1.4. Markers are per-$\mathcal{W}$ medians and the LOWESS-smoothed lines and their bands are computed on the same per-scenario medians. The bands show the across-scenario standard error. For full results see Figure A.7.

Each panel plots gap closed as a function of $\mathcal{W}$ for one prior kind, with separate lines for the three heuristics. For the **advanced**, **local random**, and **deceptive** prior kinds, gap closed peaks early, around $\mathcal{W} \approx 5$–10, and then declines as $\mathcal{W}$ grows further, most sharply for **local random**, where it falls from over $30\,\%$ to roughly $-40\,\%$ by $\mathcal{W} = 20$. Only the **expert** prior stays close to flat across the full range. The Adaptive β variant is also the clear outlier for **advanced** and **deceptive** priors, trailing the other two heuristics by a wide and consistent margin rather than merely fluctuating around them, whereas all three variants are largely indistinguishable for **expert** and **local random**, where the standard error bands overlap almost entirely. We use $\mathcal{W} = 4$ throughout the remainder of the chapter, as it sits near the early peak observed for most prior kinds while avoiding the degradation seen at larger window sizes.

5.1.6 Numerical Comparison of the Adaptive Decay Heuristics

Table 5.1 reports the area under the incumbent regret curve (AUC), the final regret, and the gap closed relative to the static linear πBO baseline, averaged over the four PD1 scenarios and broken down by prior kind. The corresponding plots on the YAHPOGym (Pfisterer et al., 2022) `rbv2_xgboost` benchmark are provided in Figure A.4. The first two metrics are

absolute, as AUC integrates a method's own incumbent regret curve over the run and final regret reports its value at the terminal iteration, so both are lowest for the best-performing variant irrespective of the oracle. Gap closed is instead relative and derived from the AOC of section 4.1. A positive value indicates the method closes part of the gap to the oracle, while a negative value indicates it performs worse than the baseline itself.

Table 5.1.: Comparison of the adaptive decay variants against static linear decay across prior kinds, reporting AUC, final regret, and gap closed relative to the oracle decay defined in subsection 3.1.3. Values are mean ± standard deviation over all scenarios and seeds, with the best value per row in **bold**.

Metric	Prior Kind	Linear	Adaptive Schedule	Adaptive β	Adaptive Schedule/β
AUC ($\times 10^{-2}$)	Expert	8.63 ± 2.40	$8.50 \pm 2.53^{\dagger}$	$\mathbf{8.48 \pm 2.49}$	8.54 ± 2.40
	Advanced	10.02 ± 2.55	10.02 ± 2.47	10.14 ± 2.57	$\mathbf{9.90 \pm 2.47}$
	Local Random	$\mathbf{10.58 \pm 2.65}$	10.60 ± 2.63	10.70 ± 2.70	10.63 ± 2.72
	Deceptive	11.65 ± 2.45	$11.33 \pm 2.41^{**}$	11.62 ± 2.55	$\mathbf{11.23 \pm 2.46}^{**}$
Final Regret ($\times 10^{-2}$)	Expert	5.84 ± 2.60	$5.57 \pm 2.64^{*}$	5.68 ± 2.71	$\mathbf{5.56 \pm 2.55}^{\dagger}$
	Advanced	7.59 ± 2.61	7.33 ± 2.48	7.65 ± 2.76	$\mathbf{7.20 \pm 2.54}^{*}$
	Local Random	8.12 ± 2.52	$\mathbf{7.90 \pm 2.67}$	8.30 ± 2.71	8.05 ± 2.68
	Deceptive	9.42 ± 2.25	$8.84 \pm 2.34^{**}$	9.48 ± 2.51	$\mathbf{8.75 \pm 2.51}^{*}$
Gap Closed (in %)	Expert	—	$24.3^{\dagger}$	$\mathbf{26.5}$	15.9
	Advanced	—	-1.3	-20.5	$\mathbf{18.4}$
	Local Random	—	$\mathbf{-3.0}$	-19.4	-7.5
	Deceptive	—	23.6^{**}	2.1	$\mathbf{31.0}^{**}$

$^{*}p < 0.05$, $^{**}p < 0.01$, $^{\dagger}p < 0.1$, Seed-paired Wilcoxon signed-rank tests against static linear πBO, computed per scenario and combined across the four PD1 scenarios.

Table 5.1 shows a clear pattern: Adaptive decay helps for **expert** and **deceptive** priors, but not consistently for **advanced** and **local random** priors. The effect is strongest for **deceptive** priors. There, Adaptive Schedule and Adaptive Schedule/β significantly reduce both AUC and final regret ($p < 0.01$), closing $23.6\,\%$ and $31.0\,\%$ of the gap. This makes sense, since a bad prior benefits from being downweighted faster than the static schedule allows. For **expert** priors the same two variants close $24.3\,\%$ and $15.9\,\%$ of the gap, though only Adaptive Schedule reaches significance. For **advanced** and **local random** priors, none of the three variants significantly beat linear decay. Adaptive β even widens the oracle gap by $20.5\,\%$ and $19.4\,\%$ for these two prior kinds. This suggests adaptivity adds little for priors that are not easily categorized. Adaptive Schedule/β is never the worst variant on **deceptive** and **advanced** priors, and closes part of the gap for three of the four prior kinds. This makes it the more robust choice overall, even though its individual gains are not always significant.

5.2 Observation-Adaptive Prior Rejection

We now turn to the rejection mechanism of Fehring et al. (2026), again using the simple regret metric introduced in section 4.1. Unlike the decay mechanism, which injects a single prior of a given quality once at $t = 13$ after initial design, the rejection mechanism now introduces repeated injections over the course of the run. We inject priors at four points, $t^{(1)} = 13$, $t^{(2)} = 23$, $t^{(3)} = 33$, $t^{(4)} = 43$, spaced ten iterations apart.

5.2.1 Analyzing the Oracle Rejection's Improvement Potential

This subsection compares the two components that constitute the oracle rejection mechanism defined in subsection 3.2.2: prior rejection at the point of injection, and retroactive rejection of priors that were initially accepted. Figure 5.7 plots both trends separately for each prior kind, averaged over all PD1 scenarios.

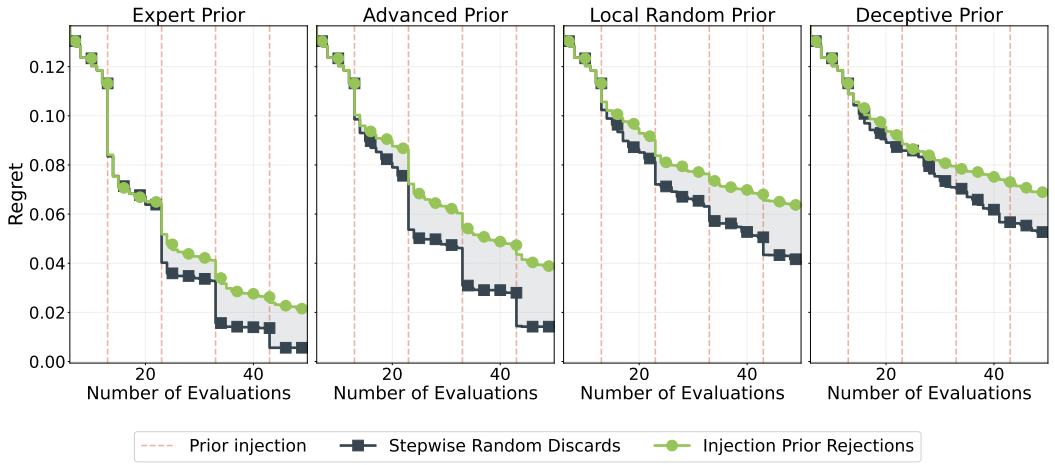


Figure 5.7.: Comparison of DynaBO, retroactively discarding already accepted priors, against DynaBO using the algorithm deciding acceptance solely at the injection points. Averaged over all PD1 scenarios and shown per prior kind.

The *injection prior rejections* curve displays the performance achievable solely through accept and reject decisions made at the point of injection, without any form of retroactive removal. The *stepwise random discards* curve, on the other hand, additionally allows discarding an already accepted prior at any later iteration, giving a stronger reference trajectory for what rejection could achieve. The gap between the two is the headroom that a stepwise mechanism could in principle exploit. We can observe a significant gap for all the plotted prior kinds in addition to less steep drops in regret at prior injection points the less informative the respective prior becomes. Furthermore, we find the regret achievable on average is lower for the **expert** and **advanced** priors compared to the other two variants, which is in line with what we expect with regard to the findings of Fehring et al. (2026).

We approximate the full rejection oracle by randomly sampling $K = 100$ discard configurations, as exhaustive evaluation is computationally infeasible. A prior injected at $t^{(i)} \in \mathcal{I}$ can

be discarded at any of the $N - t^{(i)}$ subsequent iterations, matching the range of ℓ_i defined in subsubsection 3.2.2. The total number of distinct discard configurations is therefore

$$|\mathcal{R}_{\text{full}}| = \prod_{t^{(i)} \in \mathcal{I}} (N - t^{(i)}), \tag{5.1}$$

where N is the trial count and $\mathcal{I} = \{t^{(1)}, \ldots, t^{(|\mathcal{I}|)}\}$ the set of prior injection points. For our environment with $N = 50$ and $\mathcal{I} = \{13, 23, 33, 43\}$ this results in:

$$|\mathcal{R}_{\text{full}}| = 37 \times 27 \times 17 \times 7 = 118{,}881. \tag{5.2}$$

Despite evaluating only about 0.084% of the $118{,}881$ possible discard configurations, we observe consistent improvements over the injection-only oracle. Since any sampled discard configuration yields a valid regret trajectory, the pointwise minimum over these samples serves as a lower reference for achievable regret, though, being based on a sample rather than an exhaustive search, it is not a guaranteed lower bound. The gap in regret widens as the optimization progresses and persists across all prior kinds and scenarios. Notably, the sampled oracle would change only marginally, if at all, were the $2^4 = 16$ combinations of accepting or rejecting priors at injection left out entirely: Most of the additional improvement stems from discarding priors mid-run rather than from the accept-or-reject decision taken at injection. This suggests that retroactively removing priors even after successful injection can yield substantial performance gains, motivating the stepwise rejection mechanism introduced in subsection 3.2.3.

5.2.2 First Heuristic: Stepwise Prior Rejection Check

In Figure 5.8 we compare standard prior rejection as suggested by Fehring et al. (2026) using standard linear decay with $\beta = 10$ and $\tau = -0.15$ to Vanilla BO and the oracle we analyzed in subsection 5.2.1, alongside DynaBO using Equation 3.12. The shaded area in Figure 5.8 displays the oracle we established in Equation 3.11. There is a substantially large gap between both DynaBO variants and the oracle for the **advanced** and **local random** prior kinds, as for them the heuristic of blindly accepting any prior at injection is often suboptimal, leaving considerable headroom for retroactive discards. For the **expert** prior the oracle achieves the lowest absolute final regret of all four kinds, though static DynaBO already tracks its trajectory fairly closely, since accepting a well-tuned prior at injection is close to optimal in the first place. The **deceptive** prior shows the narrowest relative gap of the four: since it should be optimal to reject a misleading prior outright at injection, there is comparatively little additional headroom for the oracle to exploit through retroactive discards. Although the gap widens substantially the further the optimization process progresses. This is notable for less informative prior kinds especially.

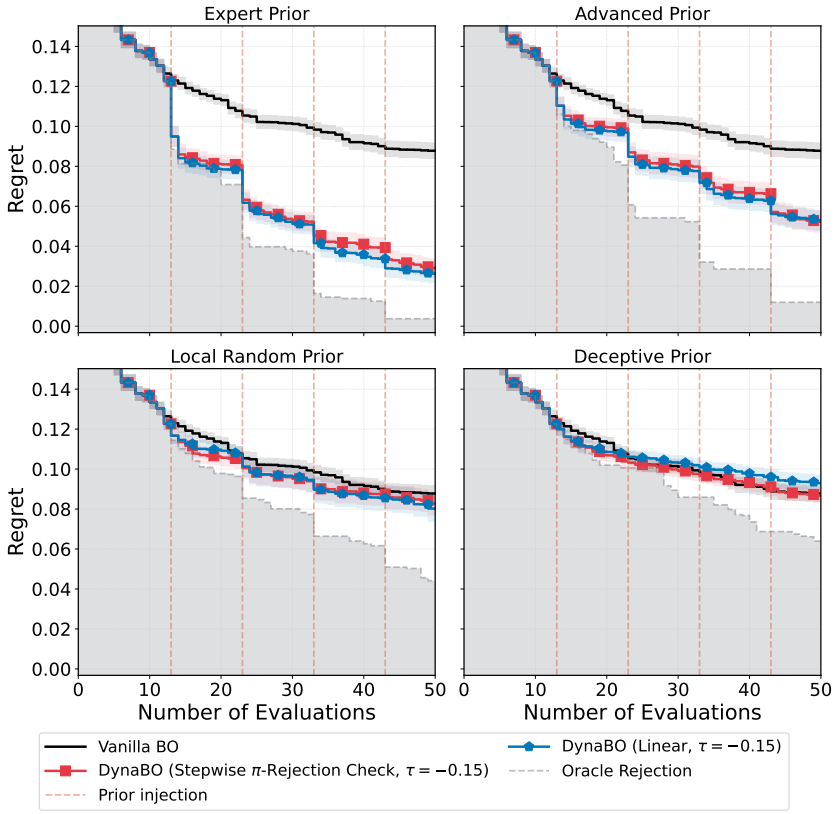


Figure 5.8.: Mean regret per prior kind for Vanilla BO, DynaBO with linear decay ($\tau = -0.15$), stepwise rejection DynaBO (subsubsection 3.2.3), and the oracle rejection of subsection 5.2.1, on `lm1b_transformer_2048`. All PD1 scenarios are shown in Figure A.8.

There is a substantial difference between Vanilla BO and the two DynaBO variants for the **expert**, **advanced**, and **deceptive** priors, confirming the findings of Fehring et al. (2026). We can additionally see the steepness in regret falloffs at each respective prior injection point is higher for more informative priors, while the **local random** and **deceptive** prior especially yield nearly smooth incumbent trajectories for all four plotted mechanisms, including the oracle rejection. The stepwise and linear rejection curves are themselves nearly indistinguishable for the **expert**, **advanced**, and **local random** priors, but visibly separate for the **deceptive** prior on `lm1b_transformer_2048`, where stepwise rejection maintains a small, consistent lead over linear DynaBO from around the second injection onward and the gap continues to widen through the fourth. While the overall improvement is hardly significant, a key takeaway is that per-step rejection checks do not worsen performance: regret stays very similar to vanilla DynaBO, with this one exception suggesting the possibility of further lowering regret through stepwise rejection.

5.2.3 Sensitivity Analysis of the Prior Rejection Threshold τ

Fehring et al. (2026) check the prior once at injection via the formula noted in Equation 2.9. Since a false classification of a harmful prior as advantageous or vice versa may lead to higher overall regret, the rejection criterion must be very strict. It needs to accept as many

expert priors as possible without starting to accept **deceptive** ones as well. Fehring et al. (2026) found $\tau = -0.15$ to strike this balance well. However, since we do not evaluate the criterion just once but instead do it on a stepwise basis, we will attempt to loosen the threshold. To find appropriate values we evaluate the sensitivity of DynaBO's performance to the rejection threshold τ.

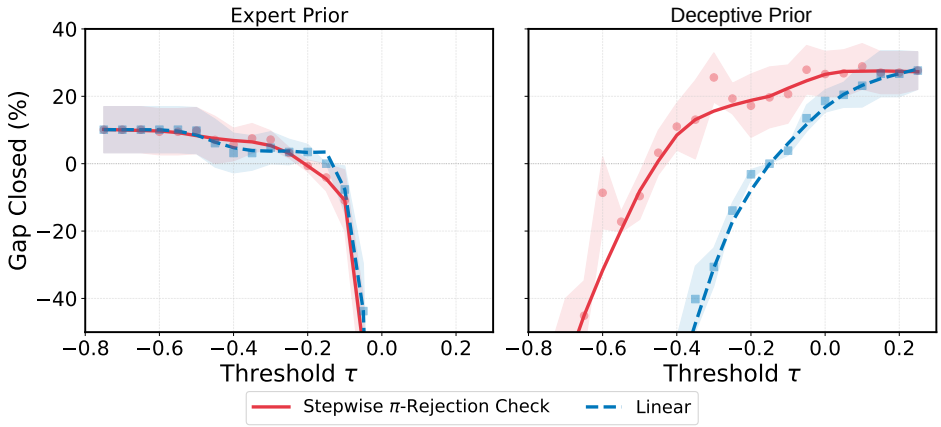


Figure 5.9.: Gap closed relative to DynaBO as a function of the rejection threshold τ for the stepwise DynaBO rejection check against a static linear schedule, shown for the expert and deceptive prior kinds. Markers are per-τ medians. Lines are LOWESS-smoothed medians across the four PD1 scenarios. The shaded band shows the across-scenario standard error ($n = 4$). For full results see Figure A.9.

As shown in Figure 5.9, the optimal choice varies substantially with prior quality. Adapting τ leads to an increase or decrease in overall prior rejections. Therefore the interesting scenarios are the ones that display regret curves of **expert** and **deceptive** priors especially. Instead of simply plotting regret we instead display the area between DynaBO using $\tau = -0.15$ and $\beta = 10$ and our rejection oracle. We then plot how much we can close this gap by adapting the value for τ for the stepwise rejection heuristic and standard DynaBO. That is why the DynaBO line labeled as *linear* crosses the x-axis at $\tau = -0.15$ in both plots. The chart is averaged over all PD1-scenarios and the shaded band shows the across-scenario standard error ($n = 4$).

It is apparent that the **expert** prior panel does not show much difference in regret regarding the time of prior rejection checks. Deciding whether to accept or reject a prior seems to work well enough when done at injection. Iterating over different thresholds does not change that and we achieve similar regret for the stepwise heuristic and vanilla DynaBO at each point. Additionally we can see a steep drop off when approaching $\tau = 0$. This is expected as that is the point where the equation starts to reject most priors regardless of their quality. This finding suggests what Fehring et al. (2026) already noted: Choosing negative values for τ is necessary for DynaBO to work well across different prior kinds.

The **deceptive** prior panel on the other hand shows a contrary perspective. When increasing values for τ both versions of DynaBO seem to close the mentioned gap quite substantially. This is an averaged decrease in the gap by over 20% seemingly converging when τ becomes larger than -0.2. However, we can also quite clearly see a huge difference in behavior regarding the prior rejection heuristic. Checking on a stepwise basis whether to retroactively

discard a prior seems to allow for substantially lower values of τ. In fact every value $\tau > -0.5$ starts to close the gap when not only rejecting priors at injection.

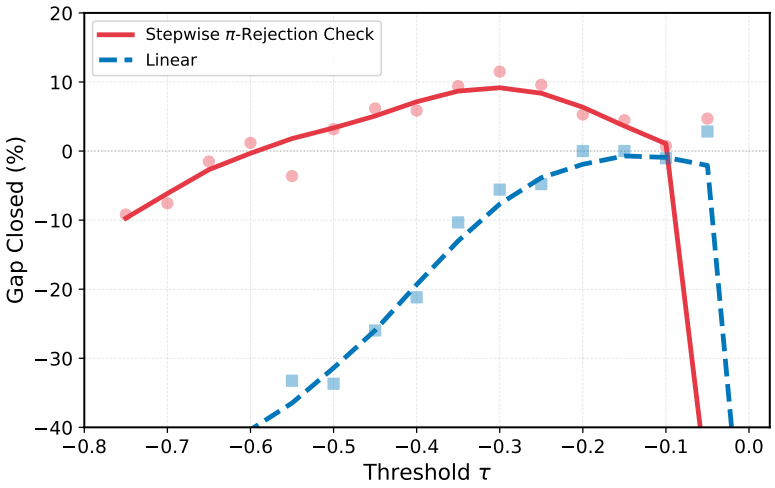


Figure 5.10.: Gap closed relative to DynaBO as a function of the rejection threshold τ, averaged over all four PD1 scenarios and prior kinds, comparing the stepwise DynaBO rejection check against a static linear schedule. Markers are per-τ medians and lines are LOWESS-smoothed.

Combining these findings we deduce that for our stepwise rejection approach τ can indeed be less strict. Figure 5.10 plots this relationship averaged over all prior kinds, and it may at first look to contradict the per-kind panels: the curve collapses sharply just past $\tau \approx -0.1$, even though the **deceptive** panel alone keeps improving toward $\tau = 0$. The collapse is driven by the **expert** and **advanced** panels being averaged in, where values near zero reject almost every prior and inflate regret. It is not a property of the **deceptive** kind. The line denoted as *linear* shows that for the expert and **deceptive** prior kinds at least, static DynaBO cannot do better than $\tau = -0.15$. The stepwise line, however, shows what we noted before: there are values of τ that are better for **deceptive** priors under stepwise rejection and therefore lower the averaged regret. The averaged curve stays positive for τ roughly between -0.6 and -0.1.

In summary τ still needs to be a negative value as to not negatively impact **expert** priors and on the other hand should not go far below -0.6 either. We therefore conclude $\tau \in [-0.6, -0.1]$ to be an effective range. In the following sections we will mostly use $\tau = -0.3$ as it seems to offer a suitable tradeoff between **expert** and **deceptive** prior kinds.

5.2.4 Second Heuristic: Stepwise Prior Rejection Check Using a Looser Threshold

Following our results from subsection 5.2.3 we will now adapt our approach to make use of looser values for τ. In Figure 5.11 we demonstrate the difference that the choice of τ has on this heuristic.

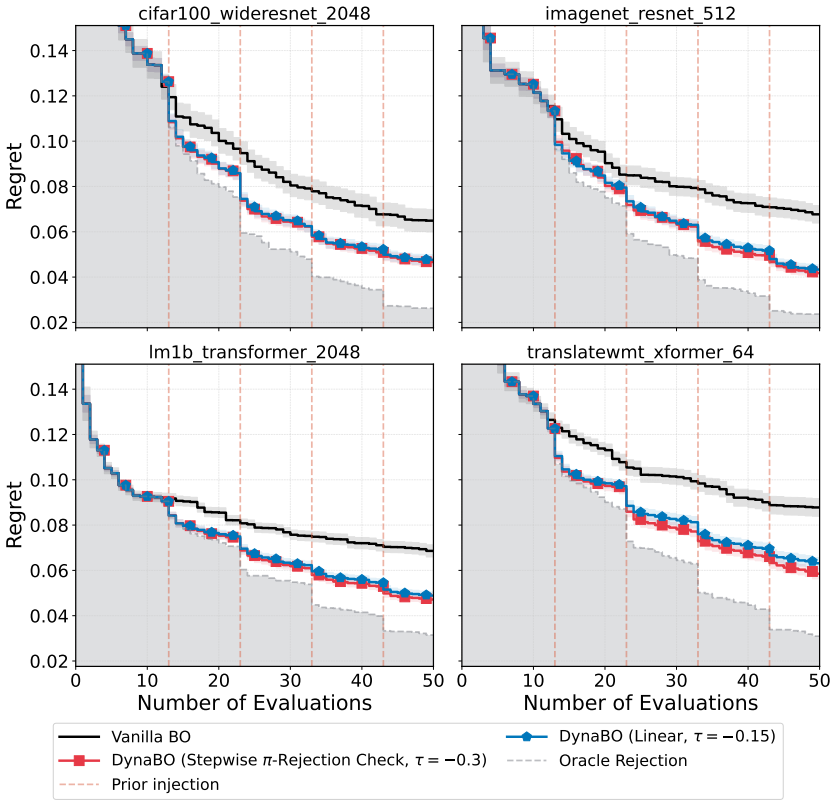


Figure 5.11.: Mean regret per PD1 scenario for Vanilla-DynaBO and stepwise rejection DynaBO (subsubsection 3.2.3) over each prior kind using $\tau = -0.3$. All PD1 scenarios are shown in Figure A.10.

Again, notice that the stepwise rejection check at $\tau = -0.3$ tracks the static linear schedule at $\tau = -0.15$ almost exactly in all four panels, regardless of scenario. This is expected once averaged over prior kind, since the two configurations only diverge noticeably for **deceptive** priors, as Figure A.10 shows, and this effect is diluted by the other three prior kinds when averaging. What does vary across panels is the size of the residual gap to the oracle: It stays comparatively narrow for **advanced** priors on `imagenet_resnet_512` and `lm1b_transformer_2048`, but remains wide throughout the run for `cifar100_wideresnet_2048` and `translatewmt_xformer_64`, indicating that unexploited headroom for stepwise rejection is scenario-dependent rather than uniform across the benchmark suite. We examine the prior-kind-specific behavior behind this averaged view, in particular the pronounced gap for **deceptive** priors, in the following section.

5.2.5 Analyzing Frequency of Post-Acceptance Prior Discards

Figure 5.12 plots the cumulative prior discards averaged over the PD1-scenarios for our stepwise prior rejection approach using $\tau \in \{-0.15, -0.5\}$.

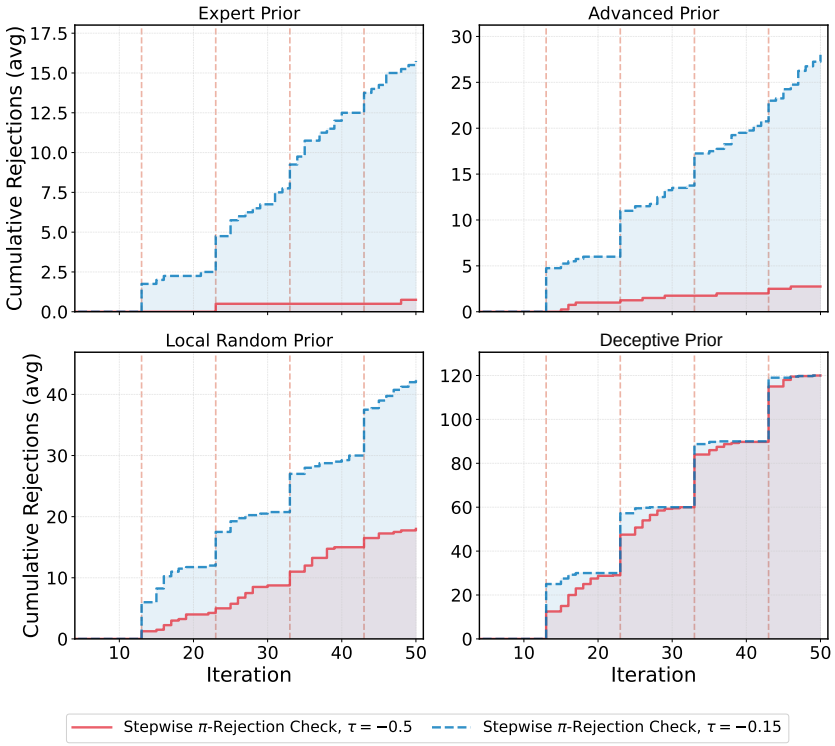


Figure 5.12.: Cumulative count of priors discarded by the stepwise DynaBO rejection check, averaged over the four PD1 scenarios and shown per prior kind, for a lenient ($\tau = -0.5$) and a strict ($\tau = -0.15$) threshold. We contrast $\tau = -0.5$ and $\tau = -0.15$, near the two ends of the effective range, rather than the adopted $\tau = -0.3$, since the discarding behavior is more legible when using an even less strict configuration (that is still reasonable however).

Remember since we ran with 30 seeds the chart can at most reject 30 priors between any 2 injection lines. We can write it more formally as:

$$|R_t| \leq |\{t_i \in \mathcal{I} \mid t_i \leq t\}| \cdot S \tag{5.3}$$

where R_t denotes the total rejections up to iteration t, $\mathcal{I}$ the set of prior injection iterations and S the number of seeds. In our case we set $\mathcal{I} = \{13, 23, 33, 43\}$ and $S = 30$.

We observe that the **expert** prior hardly discards any priors. In the end using $\tau = -0.15$ we discarded on average around 15 of 120 possible priors, or roughly 12.5%. Setting $\tau = -0.5$ on the other hand has even fewer total discards, basically none. **Advanced** priors behave similarly, although they do have more overall rejections and discards. **Local Random** priors discard even more priors, especially with $\tau = -0.15$, where the total rejection rate exceeds $\frac{1}{3}$.

Most notably however, the **deceptive** prior panel using $\tau = -0.15$ outright rejects nearly every prior at injection. Setting $\tau = -0.5$ lowers the amount of rejections on injection substantially. Instead some priors seem to get accepted at first and then all get discarded in

the following few iterations. That way, at the points just before a new prior is injected, both values for τ leave us with essentially no remaining priors.

5.3 Combined Observation-Adaptive Mechanisms

Having evaluated observation-adaptive decay and observation-adaptive rejection independently, we now examine what happens when both mechanisms are applied together. Since decay and rejection operate on different aspects of a prior's influence, one continuously reweighting it and the other deciding whether to keep it at all, combining them raises the question of whether their benefits stack, cancel, or leave one mechanism dominant. This section first evaluates the best-performing adaptive variants of both mechanisms against their respective baselines and oracles, then contrasts stepwise and injection-only rejection directly, and closes with a joint numerical evaluation across all adaptive rejection variants.

5.3.1 DynaBO with Combined Heuristics

We combine the adaptive decay schedule of subsubsection 3.1.4 with the stepwise rejection check of subsubsection 3.2.3, applying both mechanisms within a single DynaBO run rather than evaluating them in isolation as in section 5.1 and section 5.2. Figure 5.13 shows the resulting regret curves.

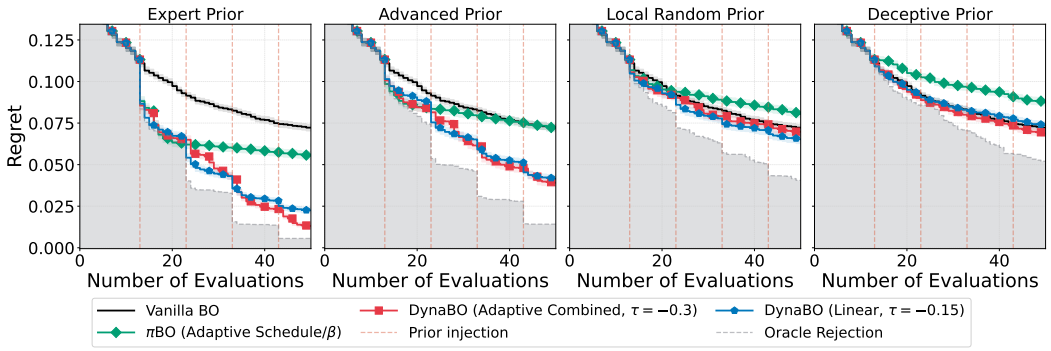


Figure 5.13.: Mean incumbent regret for Vanilla BO, the best adaptive πBO variant, and the best adaptive DynaBO variants, with the oracle rejection curve as a lower reference. It averages all PD1 scenarios per prior kind, over 30 seeds and bands show standard error. For full results see Figure A.12.

Clearly, both linear DynaBO ($\tau = -0.15$) and our adaptive combined variant ($\tau = -0.3$) outperform the best adaptive πBO heuristic. This is expected, as πBO receives a single prior whereas DynaBO receives four, so the two are not on equal footing. The gap opens immediately after the first injection, most visibly for the **expert** and **advanced** priors, and widens with each subsequent injection, since πBO receives no further priors after its single injection while both DynaBO variants keep incorporating new information. This effect is weakest for the **deceptive** prior, where the injected priors offer little useful signal to begin with, so all three methods remain closer together throughout the run. The adaptive combined variant outperforms linear DynaBO for three of the four prior kinds, though by

varying amounts: The gap stays small for **expert** and **advanced**, widening only toward the final iterations, while for **deceptive** the combined variant maintains a clearly larger and more consistent lead from around the second injection onward. **Local Random** priors are the exception, where standard DynaBO as defined by Fehring et al. (2026) performs slightly better throughout. This mirrors the pattern seen for the individual mechanisms in section 5.1 and section 5.2: Adaptivity helps most where the injected prior is least reliable, and the combined approach inherits this from its rejection component rather than losing it to added complexity.

5.3.2 Regret Difference of Stepwise and On Injection Prior Rejection Checks

The aggregate comparison in subsection 5.3.1 shows that the adaptive combined variant slightly outperforms linear DynaBO on average. However, averaging over evaluations can mask exactly when and where that advantage arises. To examine this directly, Figure 5.14 contrasts the two configurations at the level of individual iterations rather than aggregate curves, plotting the per-iteration difference in mean incumbent regret between the adaptive combined variant at $\tau = -0.3$ and the linear schedule at $\tau = -0.15$ as exemplified by Fehring et al. (2026). Green marks iterations where the combined variant has lower regret, orange the reverse. No coloring means the two approaches yield the exact same regret values, which is why the chart displays neither orange nor green trajectories for the first 12 iterations (the initial design phase) for any scenario or prior kind.

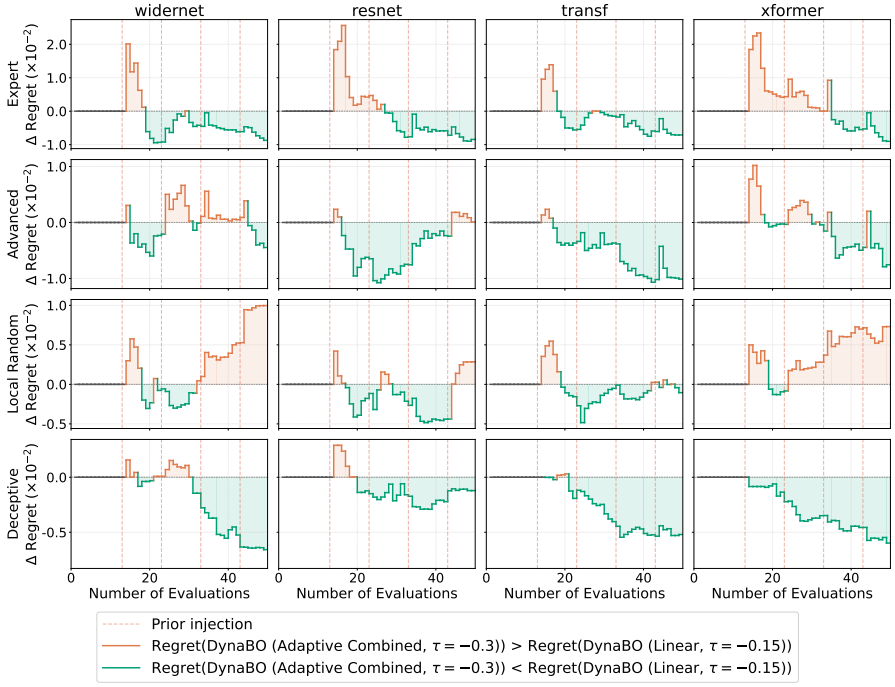


Figure 5.14.: Difference in mean incumbent regret, $\Delta = \text{Regret}(\text{adaptive combined}, \tau = -0.3) - \text{Regret}(\text{linear}, \tau = -0.15)$, per scenario (columns) and prior kind (rows). Green indicates the adaptive combined variant is better (negative Δ), orange the linear variant.

Immediately after the first injection the linear variant tends to outperform (orange) across all four prior kinds, reflecting the combined variant's more cautious acceptance of a freshly injected prior. For **expert** this pattern repeats at every subsequent injection, for **advanced** it recurs more mildly (visible in `cifar100_wideresnet_2048` and `translatewmt_xformer_64` but not `imagenet_resnet_512`), for **deceptive** it is confined to the first injection before turning and staying green almost everywhere as conservative rejection discards misleading priors sooner. For **local random** the difference trends increasingly orange over the second half of the run, most visibly in `cifar100_wideresnet_2048` and `translatewmt_xformer_64`, indicating the combined variant's cautious rejection is a poor fit for a randomly sampled, locally informative prior. Overall **deceptive** favors the combined variant strongly, **advanced** mildly, **expert** is mixed but leans toward it in later iterations, and **local random** increasingly favors the linear variant instead.

5.3.3 Evaluating the Adaptive Rejection Heuristics in Conjunction

The comparison in Table 5.2 reports the area under the incumbent regret curve (AUC), the final regret, and the gap closed relative to the static linear DynaBO baseline, averaged over the four PD1 scenarios and broken down by prior kind. The corresponding plots on the YAHPOGym (Pfisterer et al., 2022) `rbv2_xgboost` benchmark are provided in Figure A.11. As before, AUC and final regret are absolute quantities computed from a method's own incumbent regret curve, whereas gap closed is derived from the AOC against the oracle rejection. A positive value indicates the method closes part of the gap to the oracle, while a negative value indicates it performs worse than the baseline itself.

Table 5.2.: Comparison of the adaptive rejection variants against static linear DynaBO across prior kinds, reporting AUC, final regret, and gap closed relative to the rejection oracle defined in subsection 3.2.2. Values are mean ± standard deviation over all scenarios and seeds, with the best value per row in **bold**.

Metric	Prior Kind	DynaBO	$\tau{=}-0.15$ stepwise	$\tau{=}-0.3$ stepwise	$\tau{=}-0.3$ combined
AUC ($\times 10^{-2}$)	Expert	7.24 ± 1.82	7.31 ± 1.87	$\mathbf{7.17 \pm 1.81}$	7.22 ± 1.86
	Advanced	8.85 ± 2.19	8.91 ± 2.19	$\mathbf{8.70 \pm 2.11}^{\dagger}$	8.76 ± 2.36
	Local Random	9.87 ± 2.57	$\mathbf{9.81 \pm 2.54}^{*}$	9.88 ± 2.49	10.05 ± 2.72
	Deceptive	10.31 ± 2.28	$10.13 \pm 2.24^{**}$	$\mathbf{10.10 \pm 2.21}^{**}$	$10.13 \pm 2.23^{**}$
Final Regret ($\times 10^{-2}$)	Expert	2.25 ± 1.80	2.32 ± 1.90	2.08 ± 1.66	$\mathbf{1.31 \pm 1.38}^{**}$
	Advanced	4.16 ± 2.35	4.15 ± 2.33	3.94 ± 2.04	$\mathbf{3.91 \pm 2.62}$
	Local Random	6.53 ± 3.02	$\mathbf{6.42 \pm 2.92}$	6.57 ± 2.83	6.97 ± 3.26
	Deceptive	7.37 ± 2.62	$6.95 \pm 2.32^{**}$	$\mathbf{6.83 \pm 2.37}^{**}$	$6.88 \pm 2.35^{**}$
Gap Closed (in %)	Expert	—	-7.4	$\mathbf{8.3}$	3.0
	Advanced	—	-4.2	$\mathbf{10.4}^{\dagger}$	6.1
	Local Random	—	$\mathbf{5.8}^{*}$	-1.3	-17.1
	Deceptive	—	21.2^{**}	$\mathbf{25.0}^{**}$	21.7^{**}

$^{\dagger}p < 0.1$, $^{*}p < 0.05$, $^{**}p < 0.01$. Seed-paired Wilcoxon signed-rank tests against static linear DynaBO, computed per scenario and combined across the four PD1 scenarios.

On **deceptive**, all three rejection variants significantly improve both AUC and final regret over DynaBO ($p < 0.01$), with stepwise $\tau{=}{-}0.3$ performing best on every metric and reaching a gap-closed value of 25.0%. For **expert**, the combined variant achieves a lower final regret than DynaBO (1.31 vs. 2.25, $p < 0.01$), but this is not reflected in the gap-closed figures, where none of the three variants reach significance and stepwise $\tau{=}{-}0.3$ numerically leads (8.3% vs. 3.0% for the combined variant). Stepwise $\tau{=}{-}0.15$ is similar to DynaBO on AUC and final regret on **expert**, though neither difference is significant. On **advanced**, stepwise $\tau{=}{-}0.3$ yields the best AUC and gap-closed value (10.4%, $p < 0.1$), while the combined variant achieves the lowest final regret without reaching significance, so no single variant dominates uniformly for this prior kind. The **local random** results are the least favorable for the adaptive mechanisms: Stepwise $\tau{=}{-}0.15$ is the only variant that significantly improves over DynaBO, on both AUC and gap closed ($p < 0.05$), whereas the combined variant produces a gap-closed value of -17.1%, i.e. performing worse than static DynaBO itself, though this difference does also not reach significance. In summary, these results indicate that adaptive rejection yields the most consistent, statistically significant gains under strongly misleading priors, whereas for informative or locally structured priors the effect of the specific threshold and mechanism combination is mixed and mostly not statistically significant, and improvements in final regret do not necessarily coincide with improvements in gap closed.

6 Conclusion

Prior-guided Bayesian Optimization relies on parameters fixed before the run begins: πBO's decay numerator β and decay schedule d control how fast the prior fades, while DynaBO's threshold τ decides whether a prior is accepted at all. Both are heavily dependent on prior quality, a factor practitioners can hardly assess in advance. These static choices ignore information gathered during optimization and must be well tuned beforehand to avoid hindering performance, which is why this thesis proposes and evaluates observation-adaptive alternatives to these static parameters and defines oracle variants of both decay and rejection as retrospective upper bounds on what could in theory be achieved. The remainder of this chapter is structured as follows: First, section 6.1 answers the three research questions guiding this thesis and summarizes them in section 6.2. Then, section 6.3 discusses the limitations of the proposed mechanisms and their evaluation, section 6.4 outlines directions for future work, and section 6.5 closes with concluding remarks.

6.1 Answering the Research Questions

RQ1: How does πBO's fixed decay schedule compare to Vanilla BO and an oracle decay (i.e., the best possible decay from a discrete set of options) across varying prior qualities, and can observation-adaptive alternatives narrow this gap? The fixed decay schedule of πBO clearly outperforms Vanilla BO for more informative prior kinds, while conversely showing higher regret values for deceptive ones especially. Yet investigating alternative decay configurations can produce significantly lower regret values across all scenarios and prior kinds, as our oracle decay demonstrates, motivating the search for smarter mechanisms for decaying a single injected prior. The resulting gap between πBO and the oracle can be narrowed by using observation-adaptive variants, though the three variants notably differ in where they help: Adapting β alone benefits expert priors but is actively harmful for advanced and local random and leaves deceptive priors essentially unaffected, adapting the schedule d instead is more broadly effective and jointly adapting both is the most consistent, improving advanced and deceptive the most while remaining competitive on expert. Overall, jointly adapting decay strength and schedule is more robust than adapting either component alone, with local random priors remaining the one case where no adaptive mechanism closes the gap to the oracle decay. These findings are consistent with the findings of Hvarfner et al. (2022) regarding πBO's base configurations.

RQ2: How does DynaBO's fixed rejection threshold compare to Vanilla BO and an oracle threshold across varying prior qualities, and can observation-adaptive alternatives narrow this gap? Regardless of rejection threshold choice, injecting multiple priors via the configurations suggested by Fehring et al. (2026) does outperform Vanilla BO when priors are expert or advanced. For worse prior choices like local random and deceptive the method struggles a bit more, but with appropriate rejection thresholds it is

not significantly outperformed by Vanilla BO. The oracle we chose produces a large gap in regret compared to Fehring et al. (2026)'s setup, which again suggests the existence of alternative methods that exploit observations gathered mid-run to narrow this gap. As with the decay variants of RQ1, applying the rejection criterion at every step rather than only at injection, while keeping the original threshold, mainly benefits deceptive and local random priors while leaving expert and advanced essentially unaffected. We additionally found that retrospectively re-evaluating accepted priors allows for a less strict choice of τ. Doubling the threshold proposed by Fehring et al. (2026) further improves gap closed for expert, advanced, and deceptive, though it slightly underperforms the stricter threshold on local random. This relaxed stepwise threshold is therefore the strongest single-mechanism variant for rejection, benefiting three of four prior kinds while remaining acceptably performant on the fourth.

RQ3: Does combining the modified prior decay schedule and rejection criterion result in further improvement beyond what either mechanism achieves independently? Combining adaptive decay and adaptive rejection is largely compatible with the two mechanisms working independently, and in some cases benefits performance beyond either alone. For expert, advanced, and deceptive priors, the combined variant performs comparably to stepwise rejection alone, and for expert specifically it achieves the lowest final regret of any variant considered, significantly below static DynaBO. The one exception is local random, where the combined variant underperforms both standalone mechanisms and sometimes even DynaBO itself, indicating that jointly adapting decay and rejection can be counterproductive when the prior is merely local rather than genuinely poor. Overall, combining the two mechanisms rarely costs performance and can even slightly improve upon the respective heuristics its made up of. Yet local random priors remain a case where a single, more targeted mechanism is preferable to their combination.

6.2 Overall

Across all three research questions, we observe that our respective oracle constructions expose a substantial gap between static prior-related hyperparameters and what could have been achieved in hindsight. Observation-adaptive mechanisms can close a meaningful part of that gap without requiring the practitioner to commit to a fixed schedule or threshold beforehand. However, no single mechanism dominates uniformly across prior qualities. The strongest results come from the more broadly effective variants rather than the most aggressive ones. Jointly adapting decay strength and schedule for RQ1, and relaxed stepwise rejection for RQ2, each outperform their narrower counterparts. Combining decay and rejection adaptivity, by contrast, adds little beyond the better standalone mechanism and can even be counterproductive for local random priors, suggesting that stacking adaptive mechanisms is not automatically beneficial and that each should be evaluated separately. The local random priors we introduce in this thesis are the recurring exception throughout. No mechanism closes the gap to the oracle for this prior kind meaningfully, indicating that their locality and inherent randomness demand different treatment than the other more predictable prior kinds do.

6.3 Limitations

We now discuss the limitations of this thesis: Those inherited from Hvarfner et al. (2022) and Fehring et al. (2026), and those specific to our own experimental design. Building on Hvarfner et al. (2022) and Fehring et al. (2026), our approach has the same limitations: The priors are generated from known optima and therefore remain synthetically generated, regardless of the four kinds they are split into, and whether the benefits transfer to human-specified priors is left unaddressed. Beyond this inherited limitation, our own approach leaves the human-in-the-loop dimension of the problem open: Real practitioners do not draw priors from a fixed generative process but form beliefs from domain knowledge or intuition, which are harder to express experimentally. It is also unclear how a practitioner would react to the observation-adaptive mechanisms proposed here, since whether practitioners would trust or override automated prior rejection decisions is outside the scope of this thesis. Our experimental design introduces further limitations of its own: Except for a few YAHPOGym (Pfisterer et al., 2022) evaluations, all experiments used four PD1 (Wang et al., 2024) scenarios, a budget of 50 trials, and a single Random Forest surrogate, interacting significantly with our mechanisms, since $\mathcal{W} = 4$ and injections at $\{13, 23, 33, 43\}$ occupy a large fraction of the run. Both the acquisition function (EI, with LCB for the prior rejection criterion) and the surrogate were held fixed. Furthermore, the significance markers in Table 5.1 and Table 5.2 stem from dozens of pairwise tests reported without correction for multiple comparisons, so individual markers should be read as directional evidence only. Finally, both $\mathcal{W} = 4$ and $\tau = -0.3$ were selected using sweeps on the same four PD1 scenarios on which the corresponding results are reported, so the gains for these settings are reported optimistically.

6.4 Future Work

Several directions follow naturally:

- **Observation adaptivity.** This thesis used a sliding window over incumbent costs as its main signal for detecting harmful or useful priors and expanded on the LCB-difference heuristic introduced by Fehring et al. (2026). Other metrics could work just as well, such as information gain or the rate of improvement.

- **Analyzing the hyperparameter κ.** Our sliding window approach introduces the hyperparameter κ, to scale β by, in case of improvement and stagnation. We simply set it to 1.5 and 2 for Equation 3.7 and Equation 3.8 respectively, since that worked well enough for our purposes. Tuning it properly, possibly jointly with $\mathcal{W}$ and the decay ladder $\mathcal{H}$ is left unaddressed.

- **Weighted heuristics.** All our heuristics were binary. Rather than only checking *whether* something improved, one could measure *by how much* and arrive at more elegant weighted approaches to the dynamic algorithm configuration.

- **Varying the prior injection schedule.** Four injections at fixed iterations kept the oracle rejection feasible and ensured comparability with Fehring et al. (2026), but this schedule was hardcoded rather than varied. It would be interesting to explore

alternatives such as more or fewer injections, randomly sampled injection points rather than fixed ones, or mixing different prior kinds within a single run.

- **Changing the benchmark configuration.** The approaches deserve testing in other environments, including a higher trial count, a different acquisition function, a surrogate other than Random Forests, or benchmarks beyond PD1.

- **Altering the rejection criterion mid-run.** We reused the criterion of Fehring et al. (2026) unchanged, although it was designed to filter priors at injection. Re-evaluating its definition for continuous use during optimization, rather than only at injection, could prove beneficial.

- **Learned adaptation policies.** Our heuristics for adapting β, d, and the rejection decision were hand-crafted and binary. An alternative would be to learn the adaptation policy itself, either via reinforcement learning, or via an LLM-based controller that reasons over the optimization history and proposes decay or rejection decisions.

- **Adapting other hyperparameters.** There are many more hyperparameters involved we did not try to adapt in this thesis. These include but are not limited to the used acquisition function, the rejection criterion, and technically even the treshold τ itself.

6.5 Concluding Remarks

Prior-related hyperparameters require the practitioner to make a decision before the run, even though the information needed to make that decision is only revealed during the run itself. The oracle decay and oracle rejection demonstrate that lower regret values can be achieved with suitable mid-run decisions regarding decay and prior discards. Furthermore, our results show that even simple heuristics, e.g., using a sliding window to assess whether the raw costs have improved recently, can recover a meaningful amount of what the oracle shows to be achievable only in hindsight. This can be achieved reliably through stepwise rejection with a relaxed threshold and, for decay, through jointly adapting the decay-related hyperparamters β and d. These mechanisms do not significantly raise the best-case performance prior-guided Bayesian Optimization can achieve, but they substantially improve its worst-case behavior, and for a method that relies on prior beliefs whose quality cannot be assessed beforehand, raising this floor proves very valuable.

Bibliography

Carolin Benjamins, Helena Graf, Sarah Segel, Difan Deng, Tim Ruhkopf, Leona Hennig, Soham Basu, Neeratyoy Mallik, Edward Bergman, Deyao Chen, François Clément, Alexander Tornede, Matthias Feurer, Katharina Eggensperger, Frank Hutter, Carola Doerr, and Marius Lindauer (2025). *carps: A Framework for Comparing N Hyperparameter Optimizers on M Benchmarks*. Version Number: 2. DOI: 10.48550/ARXIV.2506.06143. URL: https://arxiv.org/abs/2506.06143 (visited on 2026/08/21).

Carolin Benjamins, Elena Raponi, Anja Jankovic, Koen van der Blom, Maria Laura Santoni, Marius Lindauer, and Carola Doerr (2022). *PI is back! Switching Acquisition Functions in Bayesian Optimization*. arXiv:2211.01455 [cs]. DOI: 10.48550/arXiv.2211.01455. URL: http://arxiv.org/abs/2211.01455 (visited on 2026/04/06).

Carolin Benjamins, Elena Raponi, Anja Jankovic, Carola Doerr, and Marius Lindauer (2023). *Self-Adjusting Weighted Expected Improvement for Bayesian Optimization*. arXiv:2306.04262 [cs]. DOI: 10.48550/arXiv.2306.04262. URL: http://arxiv.org/abs/2306.04262 (visited on 2026/04/06).

James Bergstra and Yoshua Bengio (2012). "Random search for hyper-parameter optimization." In: *Journal of machine learning research* 13.2.

André Biedenkapp, H Furkan Bozkurt, Theresa Eimer, Frank Hutter, and Marius Lindauer (2020). "Dynamic algorithm configuration: Foundation of a new meta-algorithmic framework". In: *ECAI 2020: 24th European Conference on Artificial Intelligence, 29 August–8 September 2020, Santiago de Compostela, Spain–Including 10th Conference on Prestigious Applications of Artificial Intelligence (PAIS 2020)*. SAGE Publications 1 Oliver's Yard, 55 City Road, London, EC1Y 1SP, pp. 427–434.

Xavier Bouthillier and Gaël Varoquaux (2020). "Survey of machine-learning experimental methods at NeurIPS2019 and ICLR2020". PhD thesis. Inria Saclay Ile de France.

Lukas Fehring, Marcel Wever, Maximilian Spliethöver, Leona Hennig, Henning Wachsmuth, and Marius Lindauer (2026). *Dynamic Priors in Bayesian Optimization for Hyperparameter Optimization*. Version Number: 3. DOI: 10.48550/ARXIV.2511.02570. URL: https://arxiv.org/abs/2511.02570 (visited on 2026/08/18).

Matthias Feurer and Frank Hutter (2019). "Hyperparameter Optimization". en. In: *Automated Machine Learning*. Ed. by Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. Series Title: The Springer Series on Challenges in Machine Learning. Cham: Springer International Publishing, pp. 3–33. ISBN: 978-3-030-05317-8 978-3-030-05318-5. DOI: 10.1007/978-3-030-05318-5_1. URL: http://link.springer.com/10.1007/978-3-030-05318-5_1 (visited on 2026/03/31).

J. C. Gower (1971). “A General Coefficient of Similarity and Some of Its Properties”. In: *Biometrics* 27.4, pp. 857–871. ISSN: 0006341X, 15410420. URL: http://www.jstor.org/stable/2528823 (visited on 2026/08/17).

Carl Hvarfner, Danny Stoll, Artur Souza, Marius Lindauer, Frank Hutter, and Luigi Nardi (2022). *πBO: Augmenting Acquisition Functions with User Beliefs for Bayesian Optimization*. Version Number: 1. DOI: 10.48550/ARXIV.2204.11051. URL: https://arxiv.org/abs/2204.11051 (visited on 2026/03/30).

Diederik P Kingma and Jimmy Ba (2014). “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980*.

Manuel López-Ibáñez and Thomas Stützle (2014). “Automatically improving the anytime behaviour of optimisation algorithms”. en. In: *European Journal of Operational Research* 235.3, pp. 569–582. ISSN: 03772217. DOI: 10.1016/j.ejor.2013.10.043. URL: https://linkinghub.elsevier.com/retrieve/pii/S0377221713008667 (visited on 2026/08/16).

Neeratyoy Mallik, Edward Bergman, Carl Hvarfner, Danny Stoll, Maciej Janowski, Marius Lindauer, Luigi Nardi, and Frank Hutter (2023). “PriorBand: Practical Hyperparameter Optimization in the Age of Deep Learning”. In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine. Vol. 36. Curran Associates, Inc., pp. 7377–7391. URL: https://proceedings.neurips.cc/paper_files/paper/2023/file/1704fe7aaff33a54802b83a016050ab8-Paper-Conference.pdf.

Florian Pfisterer, Lennart Schneider, Julia Moosbauer, Martin Binder, and Bernd Bischl (2022). “YAHPO Gym - An Efficient Multi-Objective Multi-Fidelity Benchmark for Hyperparameter Optimization”. In: *International Conference on Automated Machine Learning, AutoML 2022, 25-27 July 2022, Johns Hopkins University, Baltimore, MD, USA*. Ed. by Isabelle Guyon, Marius Lindauer, Mihaela van der Schaar, Frank Hutter, and Roman Garnett. Proceedings of Machine Learning Research. PMLR, pp. 3/1–39. URL: https://proceedings.mlr.press/v188/pfisterer22a.html.

Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando De Freitas (2016). “Taking the Human Out of the Loop: A Review of Bayesian Optimization”. In: *Proceedings of the IEEE* 104.1, pp. 148–175. ISSN: 0018-9219, 1558-2256. DOI: 10.1109/JPROC.2015.2494218. URL: https://ieeexplore.ieee.org/document/7352306/ (visited on 2026/05/03).

Artur Souza, Luigi Nardi, Leonardo B. Oliveira, Kunle Olukotun, Marius Lindauer, and Frank Hutter (2021). “Bayesian Optimization with a Prior for the Optimum”. en. In: *Machine Learning and Knowledge Discovery in Databases. Research Track*. Ed. by Nuria Oliver, Fernando Pérez-Cruz, Stefan Kramer, Jesse Read, and Jose A. Lozano. Vol. 12977. Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, pp. 265–296. ISBN: 978-3-030-86522-1 978-3-030-86523-8. DOI: 10.1007/978-3-030-86523-8_17. URL: https://link.springer.com/10.1007/978-3-030-86523-8_17 (visited on 2026/08/02).

Samuel A Stouffer, Edward A Suchman, Leland C DeVinney, Shirley A Star, and Robin M Williams Jr (1949). “The american soldier: Adjustment during army life.(studies in social psychology in world war ii), vol. 1”. In.

Zi Wang, George E. Dahl, Kevin Swersky, Chansoo Lee, Zachary Nado, Justin Gilmer, Jasper Snoek, and Zoubin Ghahramani (2024). “Pre-trained Gaussian Processes for Bayesian Optimization”. In: *Journal of Machine Learning Research* 25.212, pp. 1–83. URL: http://jmlr.org/papers/v25/23-0269.html.

Frank Wilcoxon (1945). “Individual comparisons by ranking methods”. In: *Biometrics bulletin* 1.6, pp. 80–83.

Li Yang and Abdallah Shami (2020). “On hyperparameter optimization of machine learning algorithms: Theory and practice”. en. In: *Neurocomputing* 415, pp. 295–316. ISSN: 09252312. DOI: 10.1016/j.neucom.2020.07.061. URL: https://linkinghub.elsevier.com/retrieve/pii/S0925231220311693 (visited on 2026/04/22).

Appendix

A

This appendix collects material that supports the main text without being essential to following its argument: Exact loss and regret definitions, a sensitivity check for the β value chosen for DynaBO, a full configuration heatmap for πBO's decay, and the per-scenario figures underlying the aggregated results shown in chapter 5.

A.1 Further Experimental Specifications

This section gives the precise loss and regret definitions used throughout the thesis, followed by a sensitivity analysis that justifies the choice of β for our DynaBO experiments.

A.1.1 Loss Functions

For the PD1 scenarios, the loss is the surrogate-predicted validation error at maximum fidelity:

$$\mathcal{L}(\boldsymbol{\lambda}) = \begin{cases} 1 - \mathrm{acc}_{\mathrm{val}}(\boldsymbol{\lambda}) & \text{for image tasks,} \\ \mathcal{L}_{\mathrm{CE}}^{\mathrm{val}}(\boldsymbol{\lambda}) & \text{for language tasks,} \end{cases} \tag{A.1}$$

where the image tasks comprise `cifar100_wideresnet_2048` and `imagenet_resnet_512`, and the language tasks `lm1b_transformer_2048` and `translatewmt_xformer_64`. Here $\mathrm{acc}_{\mathrm{val}}$ denotes the validation accuracy and $\mathcal{L}_{\mathrm{CE}}^{\mathrm{val}}$ the validation cross-entropy loss (Wang et al., 2024). Both metrics are exposed by `mfpbench` under a unified `valid_error_rate` interface (Wang et al., 2024; Mallik et al., 2023). Performance is measured by the simple regret after T evaluations,

$$R_T \;=\; \mathcal{L}(\hat{\boldsymbol{\lambda}}_T) \;-\; \mathcal{L}(\boldsymbol{\lambda}^*), \tag{A.2}$$

where $\hat{\boldsymbol{\lambda}}_T = \arg\min_{t \le T} \mathcal{L}(\boldsymbol{\lambda}_t)$ is the incumbent, i.e. the best configuration observed within the evaluation budget, and $\boldsymbol{\lambda}^* = \arg\min_{\boldsymbol{\lambda} \in \boldsymbol{\Lambda}} \mathcal{L}(\boldsymbol{\lambda})$ describes the surrogate's best configuration (Shahriari et al., 2016). Lower regret therefore indicates that the optimizer has found a configuration closer to the optimal one.

A.1.2 Further Beta Evaluations for DynaBO

The main text adopts $\beta = 10$ for all DynaBO experiments. The two tables below report the underlying sweep over β that motivated this choice, in terms of both area-over-curve against the rejection oracle and final regret.

Table A.1.: Area over the curve between the DynaBO incumbent and the rejection oracle of Equation 3.11, per prior kind and value of β (30 seeds, 4 scenarios, PD1). All values $\times 10^{-2}$, where smaller is better.

β	Expert	Advanced	Local Random	Deceptive	All
50	0.661 ± 0.134	1.268 ± 0.188	1.267 ± 0.225	0.902 ± 0.177	1.024 ± 0.091
20	0.821 ± 0.154	1.319 ± 0.181	1.095 ± 0.211	0.882 ± 0.176	1.029 ± 0.090
15	0.869 ± 0.143	1.419 ± 0.184	1.111 ± 0.208	0.878 ± 0.176	1.069 ± 0.089
12	0.913 ± 0.152	1.313 ± 0.179	1.031 ± 0.207	0.865 ± 0.177	1.030 ± 0.089
10	0.897 ± 0.142	1.457 ± 0.179	1.089 ± 0.204	0.872 ± 0.177	1.079 ± 0.088
7	1.014 ± 0.147	1.331 ± 0.175	1.094 ± 0.203	0.844 ± 0.175	1.071 ± 0.087
5	1.162 ± 0.150	1.506 ± 0.180	1.018 ± 0.196	0.827 ± 0.174	1.128 ± 0.088
2	1.456 ± 0.160	1.674 ± 0.162	1.142 ± 0.194	0.786 ± 0.174	1.264 ± 0.086
1	1.723 ± 0.173	1.827 ± 0.177	1.208 ± 0.204	0.752 ± 0.172	1.377 ± 0.091

Table A.2.: Final regret per prior kind and value of β (30 seeds, 4 scenarios, PD1)

β	Expert	Advanced	Local Random	Deceptive	All
50	0.017 ± 0.001	0.038 ± 0.002	0.071 ± 0.003	0.075 ± 0.002	0.050 ± 0.001
20	0.019 ± 0.002	0.040 ± 0.002	0.066 ± 0.003	0.074 ± 0.002	0.050 ± 0.001
15	0.020 ± 0.002	0.040 ± 0.002	0.067 ± 0.003	0.074 ± 0.002	0.050 ± 0.001
12	0.024 ± 0.002	0.042 ± 0.002	0.065 ± 0.003	0.073 ± 0.002	0.051 ± 0.001
10	0.023 ± 0.002	0.042 ± 0.002	0.065 ± 0.003	0.074 ± 0.002	0.051 ± 0.001
7	0.023 ± 0.002	0.041 ± 0.002	0.067 ± 0.003	0.073 ± 0.002	0.051 ± 0.001
5	0.028 ± 0.002	0.044 ± 0.002	0.064 ± 0.003	0.073 ± 0.002	0.052 ± 0.001
2	0.033 ± 0.002	0.049 ± 0.002	0.067 ± 0.003	0.072 ± 0.002	0.055 ± 0.001
1	0.040 ± 0.002	0.051 ± 0.002	0.069 ± 0.003	0.071 ± 0.002	0.058 ± 0.001

As Table A.1 and Table A.2 show, both AOC and final regret are largely insensitive to β, with the aggregate **All** column ranging only from 1.02 to 1.38×10^{-2} AOC across the tested range. Within this plateau, $\beta = 50$ and $\beta = 20$ marginally outperform $\beta = 10$ on the aggregate, so $\beta = 10$ is not the empirical optimum. We nevertheless adopt $\beta = 10$, a deliberate round doubling of the standard $N/10 = 5$, since it retains prior influence longer without sitting at the extreme of the plateau, giving the stepwise rejection mechanism more opportunity to matter.

A.2 πBO: Configuration Heatmap

The figure below complements subsection 3.1.3 by showing, for every combination of decay schedule and β, how far that fixed configuration falls behind the oracle decay, broken down per prior kind rather than aggregated.

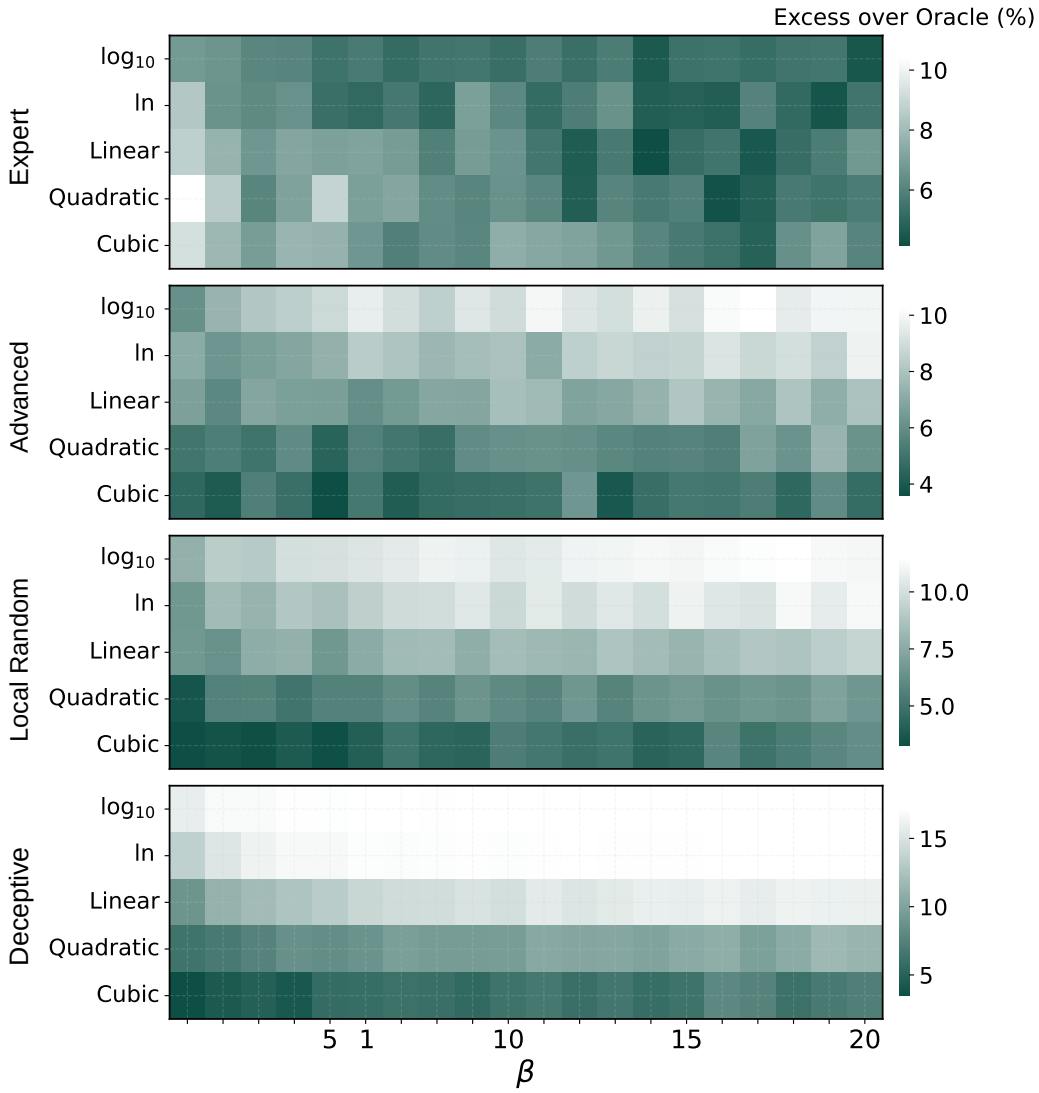


Figure A.1.: Excess over the oracle decay for every combination of decay schedule and prior strength β, shown per prior kind and averaged over all four PD1 scenarios. Each cell reports the relative gap between the area under the corresponding incumbent regret curve and that of the oracle decay defined in subsection 3.1.3. Lower values are better.

A.3 Full Adaptive Decay Results

This section provides the per-scenario counterparts to the aggregated πBO decay results discussed in chapter 5 for RQ1, covering the adaptive-schedule, adaptive-strength, and combined variants, a single-run decay trace for illustration, and the window-size sensitivity analysis.

A.3.1 Adaptive-Schedule and Adaptive-Strength Results

The per-scenario view reproduces the ordering of Figure 5.2: the adaptive schedule is indistinguishable from static linear πBO on the more informative prior kinds and separates from it most clearly for deceptive priors, though the size of that separation varies noticeably across scenarios.

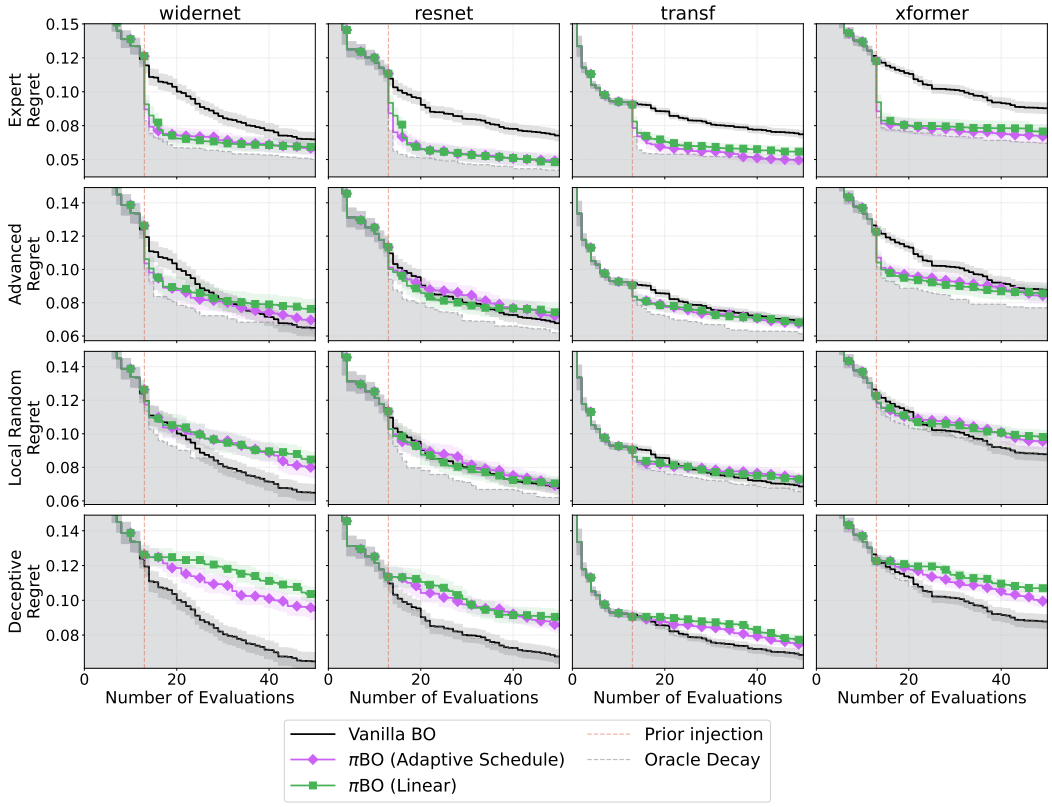


Figure A.2.: Mean incumbent regret per prior kind for Vanilla BO, Vanilla-πBO with a static linear decay, and adaptive-schedule πBO (subsubsection 3.1.4) using $\mathcal{W} = 4$, shown for all four PD1 scenarios over 30 seeds. The oracle decay of subsection 3.1.3 is included as a lower reference. `cifar100_wideresnet_2048` is reproduced in Figure 5.2.

The following trace is a single run and is included for illustration rather than as evidence. It shows that the ladder is traversed in both directions rather than saturating at $\mathcal{H}_{\min}$ or $\mathcal{H}_{\max}$, and that the exponent is driven down markedly faster for the two lower-quality prior kinds.

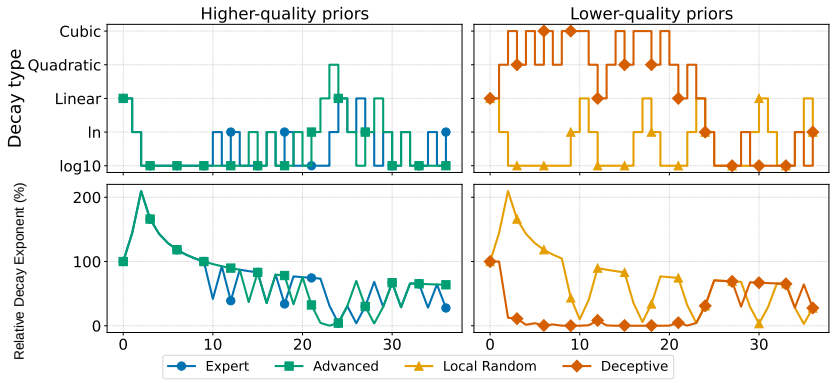


Figure A.3.: Post-injection prior impact and value current decay schedule for a single sample run on `cifar100_wideresnet_2048`.

Figure A.4 additionally reports `rbv2_xgboost` from YAHPOGym (Pfisterer et al., 2022), which serves as a check that the decay results are not specific to PD1 or to the 50-trial budget: It uses a different benchmark family, a different search space, and a budget of 200 evaluations.

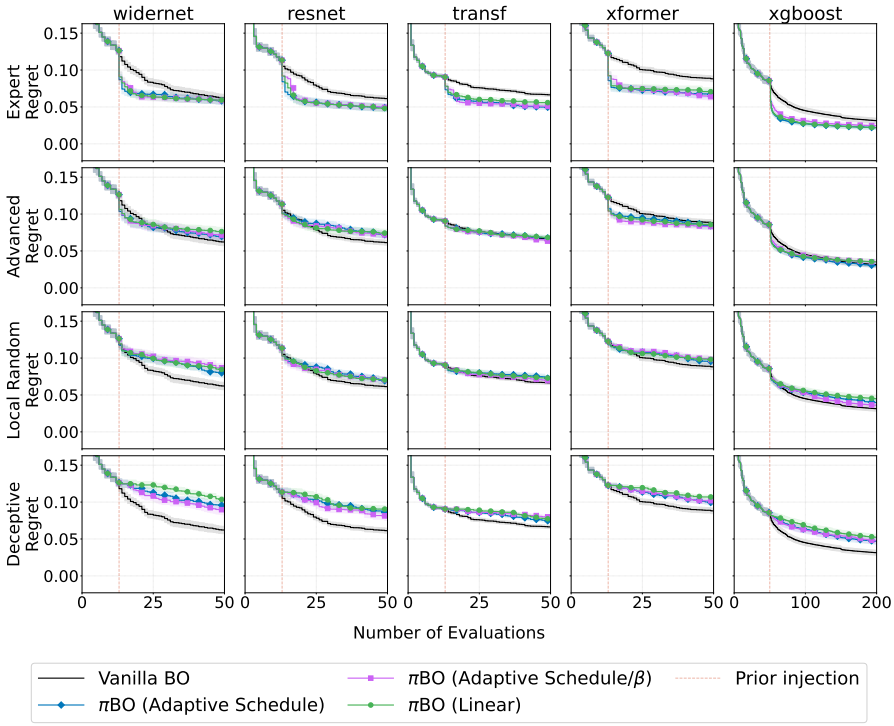


Figure A.4.: Mean incumbent regret per prior kind for Vanilla BO, static linear πBO, and the two observation-adaptive πBO variants from subsubsection 3.1.4 (adaptive schedule only, and adaptive schedule with adaptive β) using $\mathcal{W} = 4$, shown individually for all four PD1 scenarios and the YAHPOGym `rbv2_xgboost` benchmark, averaged over 30 seeds for PD1 and 10 seeds for `rbv2_xgboost`, the latter further averaged over a randomly sampled quarter of its datasets. The dashed vertical line marks the point of prior injection.

Averaged per scenario, the adaptive-strength variant is visually indistinguishable from static linear πBO across all four scenarios, which is the per-scenario counterpart to the aggregate finding that adapting β alone is insufficient.

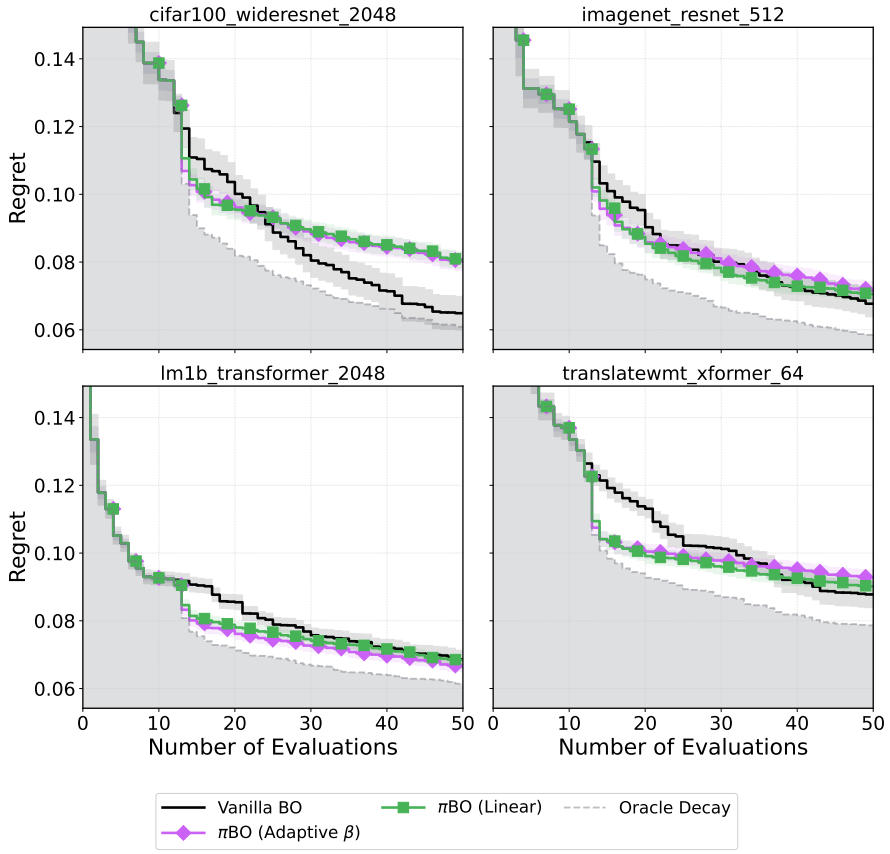


Figure A.5.: Mean incumbent regret per prior kind for Vanilla BO, Vanilla-πBO with a static linear decay, and adaptive-strength πBO (subsubsection 3.1.4) using $\mathcal{W} = 4$, shown for all four averaged PD1 scenarios over 30 seeds. The oracle decay of subsection 3.1.3 is included as a lower reference. All scenarios are shown in Figure 5.3.

A.3.2 Combined Decay Results and Window-Size Sensitivity

The remaining decay figures cover the combined heuristic and the sensitivity of all three variants to the window size $\mathcal{W}$.

The combined heuristic behaves consistently across scenarios: it matches or slightly improves on static linear πBO for the more informative kinds and produces its largest reductions for deceptive priors, with no scenario in which it is clearly worse.

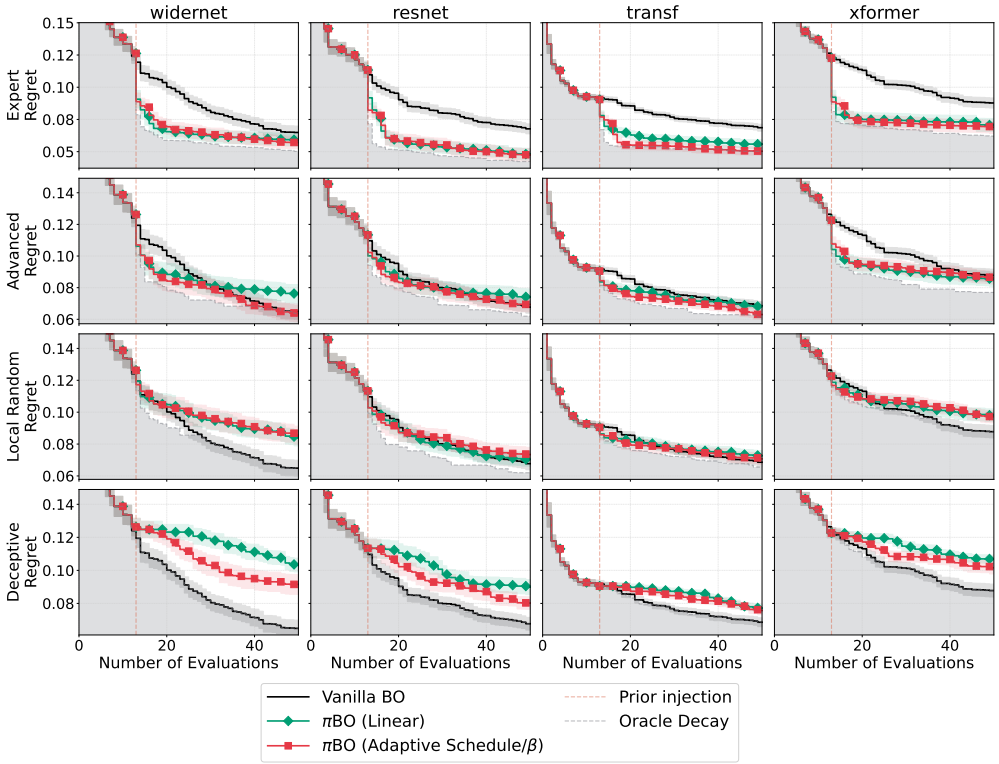


Figure A.6.: Mean incumbent regret per prior kind for Vanilla BO, Vanilla-πBO with a static linear decay, and combined adaptive πBO (subsubsection 3.1.4) using $\mathcal{W} = 4$, shown for all four PD1 scenarios over 30 seeds. The oracle decay of subsection 3.1.3 is included as a lower reference. `cifar100_wideresnet_2048` is reproduced in Figure 5.5.

Resolved per scenario, the window-size curves are considerably noisier than the aggregated view in Figure 5.6 suggests, and the early peak around $\mathcal{W} \approx 5$–10 is not equally pronounced in every scenario. The consistent feature across panels is the degradation at large $\mathcal{W}$ rather than a shared optimum, which is why we treat $\mathcal{W}$ as a parameter to be kept small rather than tuned precisely.

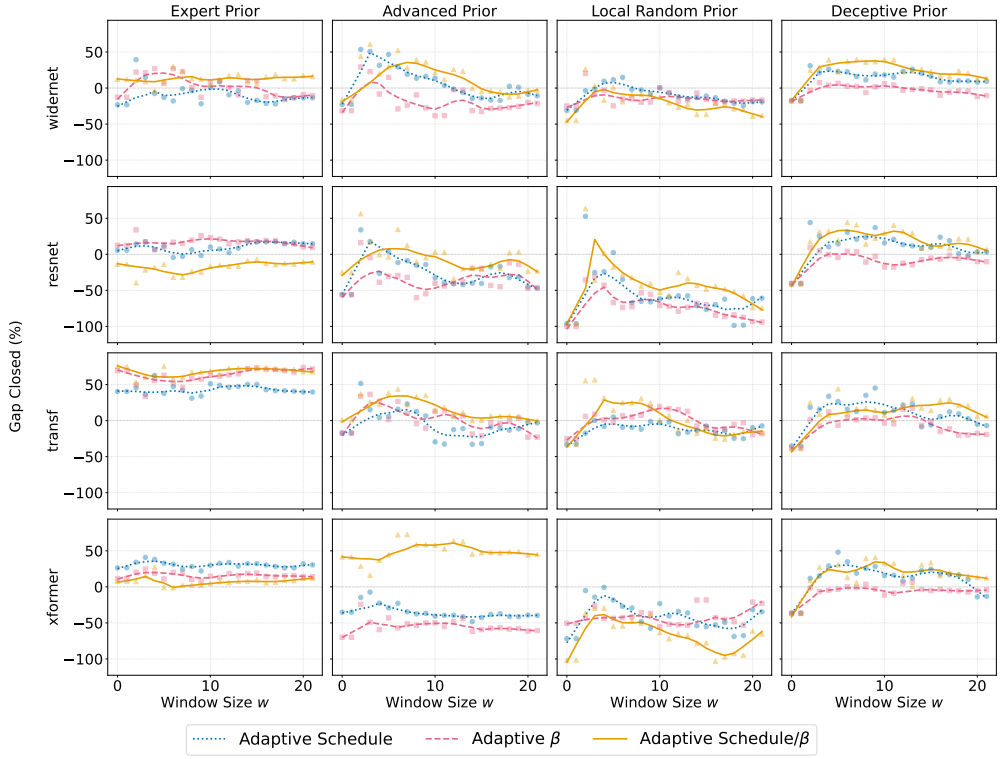


Figure A.7.: Gap closed relative to πBO with static linear decay as a function of the window size $\mathcal{W}$, resolved per PD1 scenario and prior kind, for the β-adaptive and combined schedule/β-adaptive variants. Markers are per-$\mathcal{W}$ medians and lines are LOWESS-smoothed. The scenario-averaged view is given in Figure 5.6.

A.4 Full Adaptive Rejection Results

This section gives the per-scenario and per-threshold detail behind the RQ2 results in chapter 5, first showing the stepwise rejection check at the original threshold, then sweeping the rejection threshold τ itself, and finally showing the full per-scenario comparison at the relaxed threshold adopted in the main text.

The stepwise and static curves are near-identical in most panels, with `lm1b_transformer_2048` under deceptive priors the clearest exception.

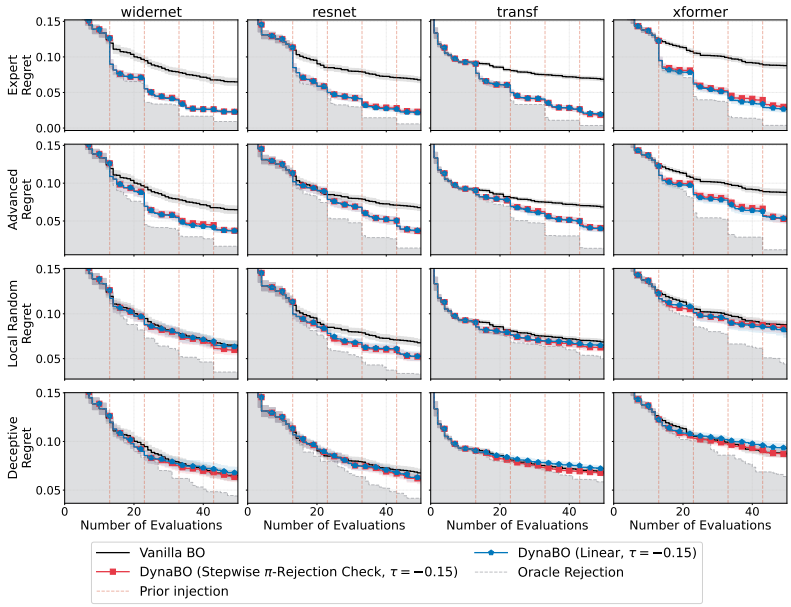


Figure A.8.: Mean incumbent regret per prior kind for Vanilla BO, Vanilla-DynaBO, and stepwise-rejection DynaBO (subsubsection 3.2.3), both at $\beta = 10$ and $\tau = -0.15$, shown for all four PD1 scenarios over 30 seeds. The shaded region marks the sampled rejection oracle of Equation 3.11. `lm1b_transformer_2048` is reproduced in Figure 5.8.

The two panels omitted from Figure 5.9 explain the collapse of the averaged curve near $\tau = 0$: Both intermediate prior kinds degrade sharply as the threshold approaches zero and almost every prior is rejected, which dominates the average even though the deceptive panel alone continues to improve.

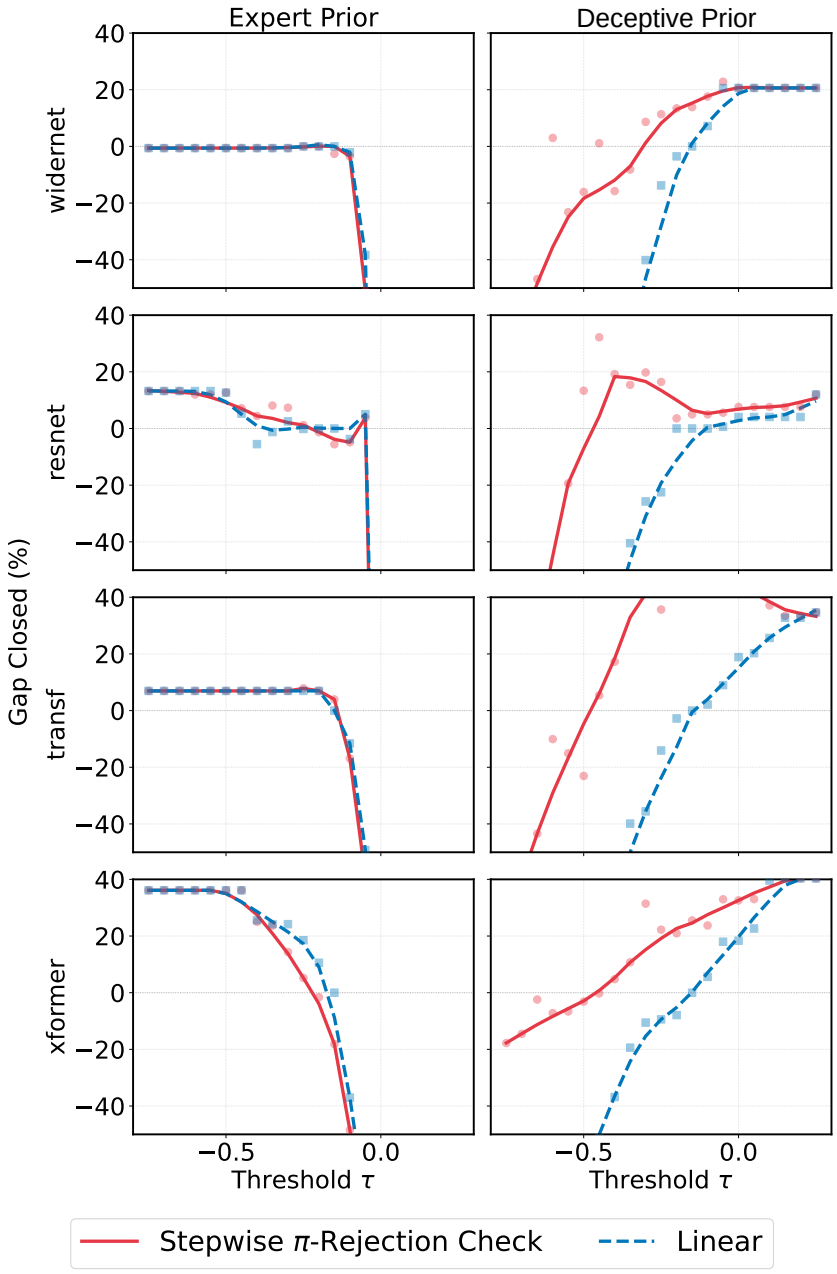


Figure A.9.: Gap closed relative to DynaBO at $\tau = -0.15$ as a function of the rejection threshold τ, shown for all four prior kinds, comparing the stepwise DynaBO rejection check against a static linear schedule. Markers are per-τ medians across the four PD1 scenarios, lines are LOWESS-smoothed, and bands show the across-scenario standard error. The expert and deceptive panels are reproduced in Figure 5.9.

At the relaxed threshold the per-scenario curves show the same behaviour as the aggregate: Essentially unchanged trajectories for the more informative prior kinds, and a visible gap to static DynaBO for deceptive priors that widens over the second half of the run.

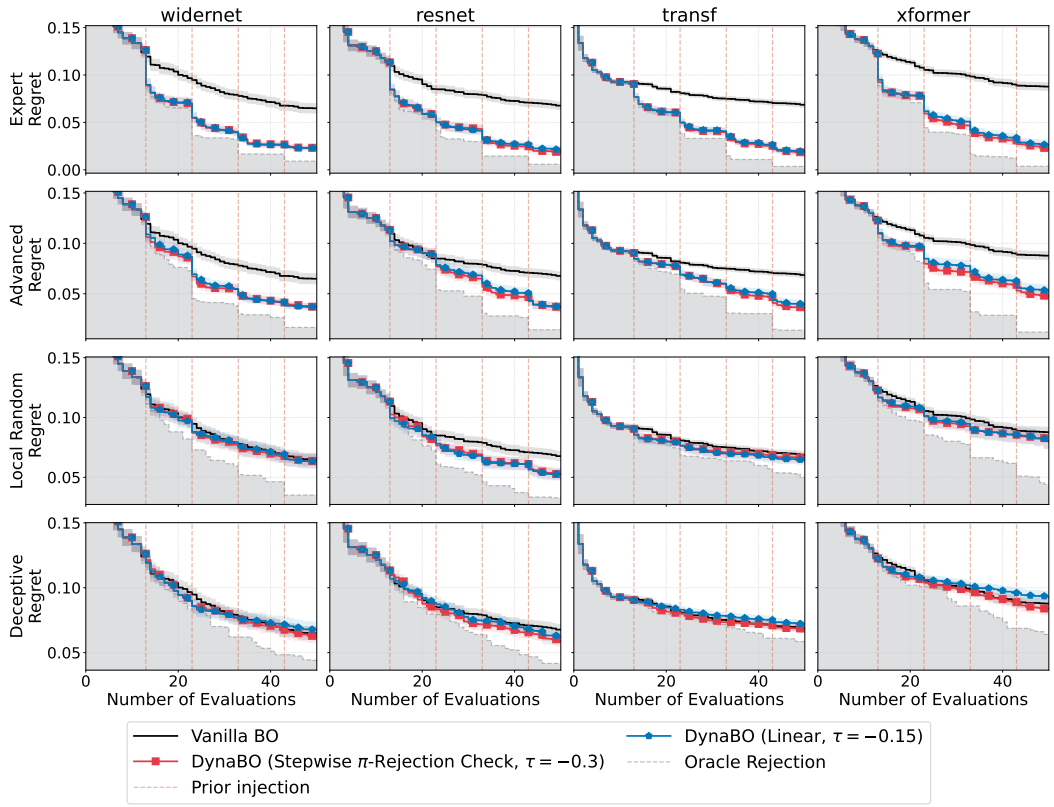


Figure A.10.: Mean incumbent regret per prior kind for Vanilla BO, Vanilla-DynaBO, and stepwise-rejection DynaBO at the relaxed threshold $\tau = -0.3$, shown for all four PD1 scenarios over 30 seeds. The prior averaged per Scenario Plot is reproduced in Figure 5.11.

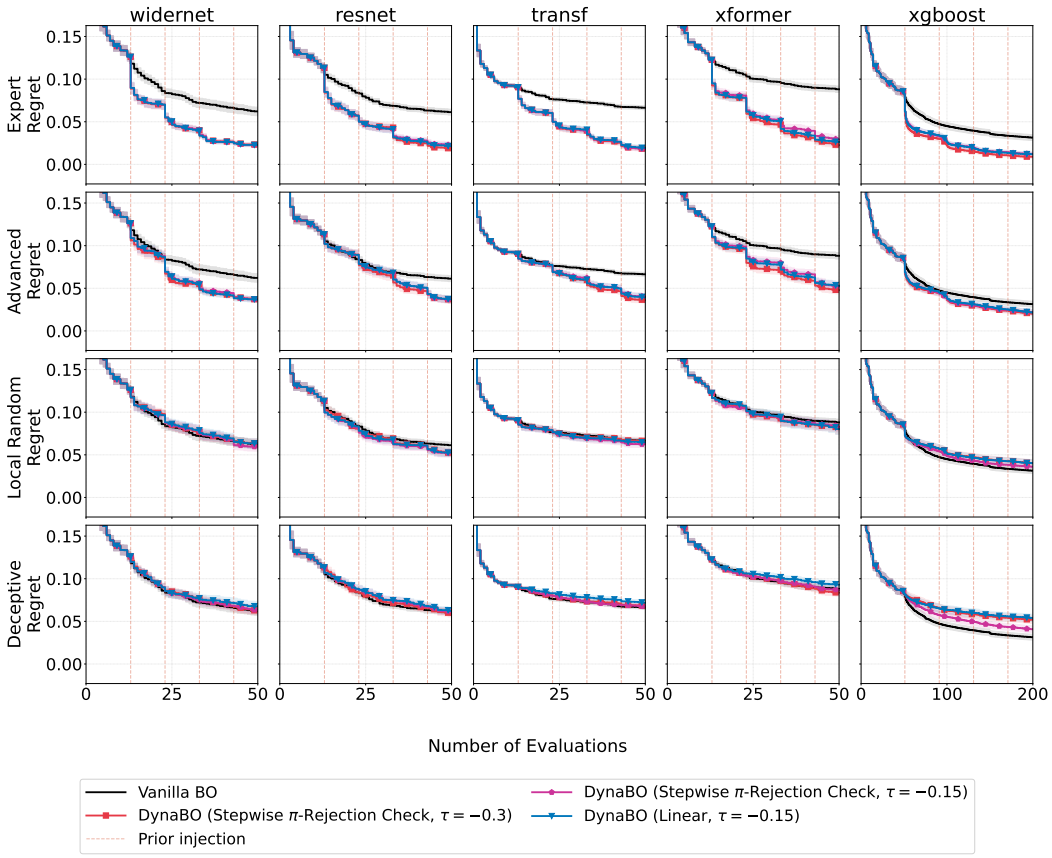


Figure A.11.: Mean incumbent regret per prior kind for Vanilla BO, DynaBO with static linear decay at $\tau = -0.15$, and the stepwise rejection DynaBO variant (subsubsection 3.2.3) at $\tau = -0.15$ and at the relaxed $\tau = -0.3$, shown individually for all four PD1 scenarios and the YAHPOGym `rbv2_xgboost` benchmark, averaged over 30 seeds for PD1 and 10 seeds for `rbv2_xgboost`. The dashed vertical lines mark the points of prior injection.

As with the decay results, we repeat the rejection comparison on `rbv2_xgboost` to check that the effect of the relaxed threshold is not confined to PD1. The relative ordering of the three DynaBO variants carries over, with the relaxed stepwise variant again strongest under deceptive priors.

A.5 Combined Mechanisms: Full Results

Finally, this section shows the per-scenario detail behind the RQ3 comparison, comparing the best-performing adaptive πBO and DynaBO variants against each other and against Vanilla BO across all four PD1 scenarios.

The per-scenario view makes the local random exception of the main text concrete: the combined variant trails static DynaBO most visibly in `cifar100_wideresnet_2048` and `translatewmt_xformer_64`, while the remaining prior kinds show the combined and stepwise variants tracking each other closely throughout.

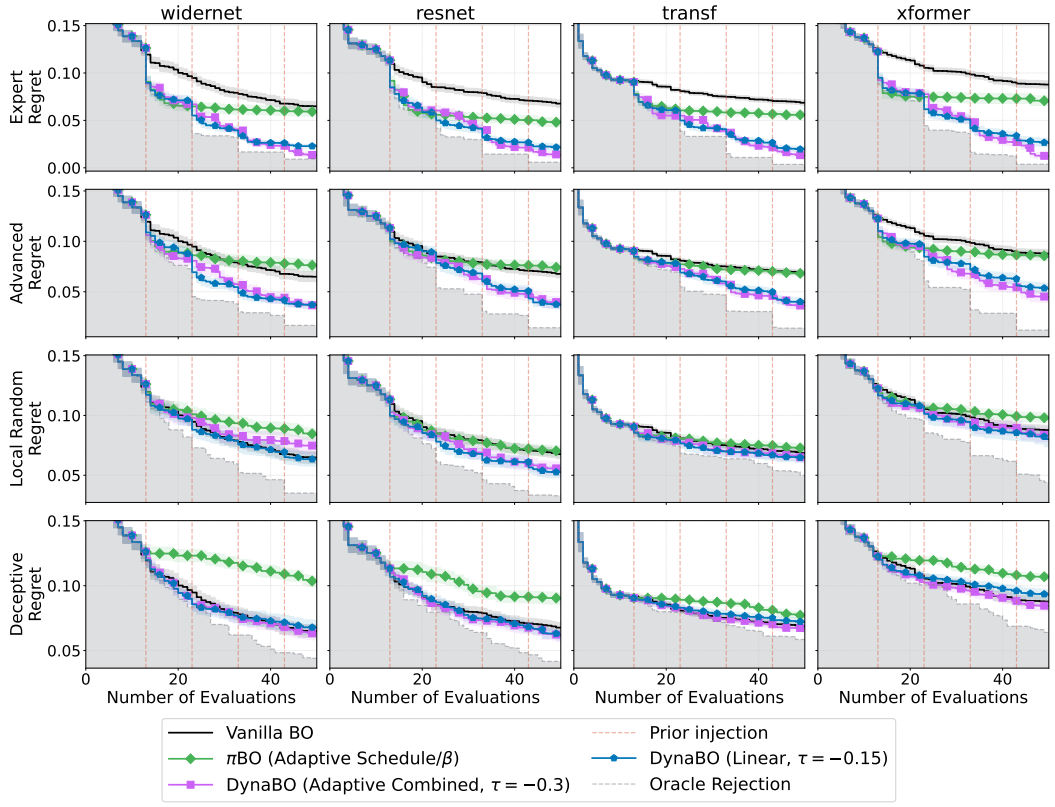


Figure A.12.: Mean incumbent regret per prior kind for Vanilla BO, the best adaptive πBO variant, and the best adaptive DynaBO variants, shown for all four PD1 scenarios over 30 seeds with bands giving the standard error. The oracle rejection curve serves as a lower reference. Prior Kind Average is reproduced in Figure 5.13.

Colophon

This thesis was typeset with LaTeX 2_ε. It uses the *Clean Thesis* style developed by Ricardo Langner.

Download the *Clean Thesis* style at `http://cleanthesis.der-ric.de/`.